

Guide de préparation à la soutenance — PlatformServerless

Guide de préparation à la soutenance

Avertissement de lecture

Partie I — Fiche d'identité du projet

Les chiffres à connaître par cœur

Les 15 entités du modèle de données **[CODE]**

Les 15 contrôleurs REST **[CODE]**

Architecture en une phrase

Partie I bis — Problématique, existant et valeur ajoutée

Q-A1 — Quelle est votre problématique ?

Q-A2 — Quelle est votre étude de l'existant ? Pourquoi ne pas avoir pris une solution existante ?

Q-A3 — Quelle est votre solution proposée, en une réponse ?

Q-A4 — Quelle est la valeur ajoutée de votre plateforme ?

Q-A5 — Qu'est-ce qu'une architecture événementielle, et pourquoi en avoir fait une ?

Q-A6 — **[EXPERT]** Où vit le type d'un événement ? Qui le déclare ?

Q-A7 — Comment un Trigger est-il relié à son Broker ?

Q-A8 — **[PIÈGE]** Vos microservices passent-ils par votre API pour publier ?

Q-A9 — Pourquoi Kafka et pas RabbitMQ ?

Q-A10 — Comment votre plateforme se compare-t-elle à Cloud Run aujourd'hui ?

Partie II — Questions par sprint

Sprint 0 — Mise en place de l'infrastructure Kubernetes

1. Compréhension du sprint

Q0.1 — Pourquoi Kubernetes plutôt que Docker Compose ?

Q0.2 — Expliquez le rôle de chaque composant du control-plane

Q0.3 — Pourquoi Cilium, et pas Calico ou Flannel ?

Q0.4 — **[PIÈGE]** Un namespace garantit-il l'isolation entre deux tenants ?

Q0.5 — Pourquoi MetalLB ? Et comment une requête externe atteint-elle un Pod ?

Q0.6 — **[EXPERT]** Votre convention `sslip.io` est-elle viable en production ?

Sprint 1 — Déploiement de Knative Serving

1. Compréhension du sprint

Q1.1 — Pourquoi Knative plutôt que des Deployments Kubernetes classiques ?

Q1.2 — **[EXPERT]** Tracez le cycle complet d'un scale-to-zero et d'un réveil

Q1.3 — **[AVANCÉ]** Pourquoi Fabric8 plutôt que d'exécuter `kubectl` ?

Q1.4 — **[EXPERT]** Que fait exactement le `KnativeWatcher` ?

Sprint 2 — Mise en place de Kafka et Strimzi

Q2.1 — Expliquez les notions fondamentales de Kafka

Q2.2 — **[AVANCÉ]** Montrez-moi où Kafka est configuré dans votre code

Q2.3 — **[AVANCÉ]** Pourquoi AdminClient et pas Fabric8, alors que Strimzi expose une CRD `KafkaTopic` ?

Q2.4 — **[PIÈGE]** Kafka garantit qu'aucun message n'est perdu ?

Q2.5 — Pourquoi Strimzi ? Qu'est-ce qu'un opérateur ?

Sprint 3 — Gestion des utilisateurs et des accès

Q3.1 — **[EXPERT]** Quel flow OAuth2 utilisez-vous exactement ?

Q3.2 — Comment un JWT est-il validé ? Keycloak est-il appelé à chaque requête ?

Q3.3 — Différence entre rôle et permission dans votre modèle ?

Q3.4 — **[EXPERT]** Si je change l'ID dans l'URL, que se passe-t-il ?

Q3.5 — Comment un CLIENT_ADMIN ajoute-t-il un membre ? Y a-t-il un e-mail d'invitation ?

Q3.6 — Que se passe-t-il si quelqu'un s'inscrit via le formulaire public ?

Q3.7 — Comment sont stockées les clés d'API ?

Sprint 4 — Gestion des applications

Q4.1 — **[EXPERT]** Tracez tout ce qui se passe quand un CLIENT_ADMIN déploie une application

Q4.2 — **[EXPERT]** Le piège du proxy Spring : pourquoi `AppDeploymentAsyncRunner` est-il une classe séparée ?

Q4.3 — Comment fonctionne le retour à une révision antérieure ?

Q4.4 — **[AVANCÉ]** Pourquoi une application supprimée reste-t-elle en base ?

Q4.5 — Comment détectez-vous qu'une application cliente plante en boucle ?

Sprint 5 — Gestion de la messagerie événementielle

Q5.1 — **[EXPERT]** Un événement est publié. Tracez son chemin jusqu'à l'exécution du code client

Q5.2 — Pourquoi passer par un Broker ? Pourquoi pas de Kafka directement vers l'application ?

Q5.3 — Le câblage Kafka au déploiement, comment ça marche ?

Sprint 6 — Gestion de l'observabilité applicative

Q6.1 — Pourquoi SSE et pas WebSocket ou polling ?

Q6.2 — D'où viennent les métriques ?

Sprint 7 — Gestion de la facturation et des paiements

Q7.1 — Comment calculez-vous la facture d'un client ?

Q7.2 — **[EXPERT]** Comment savez-vous qu'un webhook Stripe vient réellement de Stripe ?

Sprint 8 — Comptes clients, quotas et audit

Q8.1 — Comment empêchez-vous un client de consommer tout le cluster ?

Q8.2 — Que trace exactement votre journal d'audit ?

Q8.3 — Techniquement, que fait la suspension d'un client ?

Sprint 9 — Exploitation du cluster et statut public

Q9.1 — **[PIÈGE]** Pourquoi `/api/status` est-il public alors que `/api/admin/cluster/**` est protégé ?

Sprint 10 — Durcissement et sécurité

Q10.1 — **[EXPERT]** Expliquez votre RBAC. Le principe du moindre privilège est-il respecté ?

Q10.2 — Expliquez votre NetworkPolicy

Q10.3 — Cilium est-il un mécanisme d'authentification ?

Sprint 11 — Pipeline CI/CD complet

Q11.1 — Un développeur pousse un commit. Que se passe-t-il ?

Q11.2 — **[AVANCÉ]** Pourquoi Kaniko plutôt que Docker-in-Docker ?

Q11.3 — **[PIÈGE]** Pourquoi déployer avec le tag versionné et pas `latest` ?

Q11.4 — **[EXPERT]** Où sont vos secrets, et comment sont-ils protégés ?

Partie III — Questions transversales

Q-T1 — **[EXPERT]** Pourquoi dites-vous que votre plateforme est Cloud Native ?

Q-T2 — **[EXPERT]** Où se situe la frontière de confiance de votre architecture ?

Q-T3 — **[EXPERT]** Quel est le principal point de défaillance unique ?

Q-T4 — Comment votre architecture évolue-t-elle de 10 à 1 000 clients ?

Q-T5 — Comment prouveriez-vous que deux tenants sont réellement isolés ?

Q-T6 — Où est votre dette technique ?

Q-T7 — Que se passe-t-il si... (scénarios de panne)

Q-T8 — Questions de debugging

Partie IV — 30 questions pièges

Partie V — Réponses orales à mémoriser

« Présentez votre projet en une minute »

« Présentez votre architecture »

« Quel est votre apport personnel ? »

« Votre choix technique le plus important ? »

« Votre plus grande difficulté technique ? »

« Si vous refaisiez le projet, que changeriez-vous ? »

« Expliquez la sécurité de votre plateforme »

Partie VI — Tableaux de synthèse

Toutes les technologies

Checklist « je sais expliquer »

Les 12 questions à préparer en priorité

Trois principes pour le jour J

Guide de préparation à la soutenance

PlatformServerless — PaaS serverless multi-tenant

Adel Bettaieb · PFE ESPRIT · NextStep IT

Avertissement de lecture

Ce guide n'est pas un cours sur Kubernetes. C'est **la liste des questions qu'on peut te poser sur TON projet, avec les réponses que TU dois donner.**

Chaque affirmation technique a été vérifiée dans le code source, les fichiers de configuration et les manifestes Kubernetes du dépôt. Quand une information n'a pas pu être confirmée, c'est écrit.

Code de niveau

Symbole	Niveau
[BASE]	Base — tu dois répondre sans hésiter
[INTERMÉDIAIRE]	Intermédiaire
[AVANCÉ]	Avancé
[EXPERT]	Expert
[PIÈGE]	Piège — le jury cherche une contradiction

Statut de chaque information

Marqueur	Signification
[CODE]	Vérifié dans le code source ou la configuration
[MÉMOIRE]	Présent dans le mémoire, non reconfirmé dans le code
[THÉORIE]	Explication générale de la technologie
[NON IMPLÉMENTÉ]	Connu, mais absent du projet
[NON VÉRIFIÉ]	Non confirmable

La règle d'or de l'oral

Ne réponds jamais uniquement par de la théorie. Le jury sait ce qu'est Kafka. Ce qu'il veut savoir, c'est **ce que TU en as fait.**

Structure de toute bonne réponse :

Ce que c'est (1 phrase) → Comment nous l'utilisons (le cœur) → Pourquoi ce choix → Sa limite chez nous

Partie I — Fiche d'identité du projet

Les chiffres à connaître par cœur

Élément	Valeur	Source
Nœuds du cluster	3 (1 control-plane, 2 workers)	[CODE]
Version Kubernetes	v1.30.3	[CODE] capture
OS / runtime	Ubuntu 24.04.4 LTS, containerd 2.2.1	[CODE] capture
CNI	Cilium 1.19.1	[CODE] capture
Opérateur Kafka	Strimzi 0.45.0	[CODE] capture
Java	21 (<code><java.version>21</java.version></code>)	[CODE] pom.xml
Spring Boot	3.2.3	[CODE] pom.xml
Client Kubernetes	Fabric8 kubernetes-client 6.10.0	[CODE] pom.xml
Stripe	stripe-java 25.3.0	[CODE] pom.xml
Port du backend	8082	[CODE] application.yml
Rôles	ADMIN, CLIENT_ADMIN, MEMBER	[CODE] UserRole.java
Permissions granulaires	8	[CODE] Permission.java
Actions auditées	8	[CODE] AdminAction.java
Entités JPA	15	[CODE]
Contrôleurs REST	15	[CODE]
Quotas par défaut	2000m CPU, 4 Gio, 10 applications	[CODE] QuotaService
Tarif CPU	0,048 \$/vCPU/heure	[CODE] BillingService
Tarif mémoire	0,006 \$/Go/heure	[CODE] BillingService
Rate limit statut public	30 requêtes / 60 s / IP	[CODE] StatusRateLimitFilter
Image backend	adelbettaieb/platform-api	[CODE] Jenkinsfile.backend

Les 15 entités du modèle de données [CODE]

User, App, ApiKey, KafkaTopic, KafkaSource, Trigger, DeploymentLog, Metric, BillingSnapshot, AppInvoice, PaymentTransaction, TenantQuota, AdminAuditLog, Incident, DockerImage

| Les 15 contrôleurs REST [CODE]

`AuthController`, `UserController`, `TeamController`, `ApiKeyController`, `AppController`,
`KafkaController`, `EventingController`, `EventController`, `LogController`, `MetricsController`,
`BillingController`, `InvoiceController`, `PaymentController`, `AdminController`,
`StatusController`

| Architecture en une phrase

Un backend Spring Boot qui traduit des demandes métier en ressources Kubernetes, via Fabric8, en isolant chaque client dans son propre namespace, avec exécution serverless par Knative et messagerie par Kafka.

Partie I bis — Problématique, existant et valeur ajoutée

Ces questions arrivent **au tout début de la soutenance**, souvent avant la moindre question technique. Elles décident de la première impression. Un candidat qui bafouille ici part avec un handicap, même si la suite est excellente.

| Q-A1 — Quelle est votre problématique ?

Niveau : [BASE] · **Domaine :** Cadrage

Ce que le jury vérifie : que tu sais formuler un problème, pas seulement décrire une solution.

Réponse courte à l'oral : > Déployer une application sur Kubernetes exige une expertise pointue : namespaces, autoscaling, révisions, isolation réseau entre locataires. C'est un frein réel pour une équipe qui veut simplement mettre son application en ligne. Par ailleurs, les modèles de facturation classiques — instances réservées, forfait — sont inadaptés à des charges intermittentes, où une application reste inactive une grande partie du temps. > > D'où la question : comment permettre à des clients multiples de déployer et d'exploiter des applications conteneurisées de manière autonome et sécurisée, avec une mise à l'échelle automatique incluant le retour à zéro instance, une intégration événementielle native, et une facturation reflétant l'usage réel ?

Le contexte à donner si on te demande « pourquoi ce projet chez NextStep IT ? » : L'entreprise propose aujourd'hui de l'IaaS — des ressources virtualisées brutes. La tendance du marché est d'ajouter des couches d'abstraction pour que les équipes se concentrent sur leurs applications. Ce projet est cette montée en gamme : passer de l'IaaS au PaaS. **[MÉMOIRE]**

Erreur à éviter : commencer par « j'ai fait une plateforme avec Kubernetes et Knative ». C'est la solution, pas la problématique. Le jury veut d'abord le problème.

| Q-A2 — Quelle est votre étude de l'existant ? Pourquoi ne pas avoir pris une solution existante ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Cadrage

Ce que le jury vérifie : que tu n'as pas réinventé la roue par ignorance.

Réponse structurée — quatre familles, quatre défauts :

Solution	Ce qu'elle apporte	Pourquoi elle ne convient pas ici
Heroku	Expérience de déploiement très simple (git push)	Propriétaire et fermée, peu de contrôle sur l'infrastructure, coûts croissants à l'échelle
AWS Lambda / Cloud Run / Azure Container Apps	Excellente mise à l'échelle managée	Verrouillage fournisseur fort, et impossible à héberger sur une infrastructure privée — rédhibitoire pour un intégrateur
OpenShift	PaaS complet sur Kubernetes, robuste	Licence commerciale, coût d'entrée élevé, lourd à opérer — incompatible avec une offre destinée à des PME
Knative seul	Scale-to-zero et modèle événementiel, open source, déployable partout	Brique d'infrastructure, pas une plateforme : ni portail, ni multi-tenance, ni facturation, ni administration

La conclusion à énoncer clairement : > Il existe un vide entre des briques open source performantes mais inachevées — Knative, Strimzi — et des plateformes commerciales complètes mais fermées ou coûteuses. Notre projet occupe précisément cet espace : construire, **au-dessus** de briques open source éprouvées, la couche produit qui leur manque.

Relance probable : « Vous n'avez donc rien inventé ? »

J'ai assemblé des briques existantes, et c'était le bon choix — réimplémenter Knative ou Kafka aurait été absurde. Ce que j'ai construit, c'est la couche qui n'existe dans aucune de ces briques : le modèle multi-tenant, le portail, la facturation à l'usage, l'administration. Knative sait exécuter un conteneur en serverless ; il ne sait pas qu'un client existe, ni qu'il doit être facturé.

Q-A3 — Quelle est votre solution proposée, en une réponse ?

Niveau : [BASE] · **Domaine :** Cadrage

Réponse — annonce d'abord la structure, puis déroule :

La solution s'articule autour de neuf composants, que je regroupe en quatre couches.

Le socle : un cluster Kubernetes où chaque client dispose de son propre namespace, isolé par NetworkPolicy.

L'exécution : Knative Serving pour faire monter les applications en charge et redescendre à zéro ; Kafka opéré par Strimzi comme bus d'événements ; Knative Eventing pour relier les deux, avec Broker, Trigger et KafkaSource, ce qui permet de réveiller une application à zéro instance.

La couche produit : un backend Spring Boot qui orchestre le cluster via Fabric8 et porte toute la logique métier ; deux portails React, un pour les clients, un pour l'exploitant ; Keycloak pour l'identité et les rôles ; PostgreSQL pour la persistance.

L'outillage interne : une chaîne CI/CD Jenkins et Kaniko, et une supervision Prometheus.

Le piège à devancer : précise que la CI/CD est **interne à l'équipe**, pas un service offert aux clients de la plateforme. C'est écrit ainsi dans ton mémoire, et un jury pourrait croire que tu proposes du CI/CD à tes clients. **[MÉMOIRE]**

| Q-A4 — Quelle est la valeur ajoutée de votre plateforme ?

Niveau : [INTERMÉDIAIRE] · **Domaine** : Métier

Ce que le jury vérifie : que tu penses produit, pas seulement technique.

Réponse — trois valeurs distinctes selon à qui l'on parle :

Pour le client final — il déploie une application en fournissant une image Docker, un port et des bornes de mise à l'échelle. Il ne voit jamais un namespace, une NetworkPolicy, un Ingress ni une révision. La barrière d'entrée Kubernetes disparaît.

Pour NextStep IT — l'offre passe de l'IaaS au PaaS, donc d'une facturation de ressources brutes à un service à plus forte valeur ajoutée. Et la plateforme tourne sur son **propre** cluster : pas de dépendance à un hyperscaler, cohérent avec le positionnement d'intégrateur.

Sur le plan économique — la facturation suit l'usage réel, pondérée par un facteur de disponibilité. Une application inactive ne coûte quasiment rien. C'est ce que le scale-to-zero rend possible : sans lui, ce modèle de facturation n'aurait aucun sens.

La phrase qui résume : > La valeur ajoutée n'est pas dans les briques — elles sont open source et accessibles à tous. Elle est dans **l'assemblage** : transformer des briques d'infrastructure en un produit exploitable commercialement, avec un modèle multi-tenant sûr, une facturation cohérente et une interface utilisable sans expertise Kubernetes.

| Q-A5 — Qu'est-ce qu'une architecture événementielle, et pourquoi en avoir fait une ?

Niveau : [INTERMÉDIAIRE] · **Domaine** : Architecture

Ce que le jury vérifie : que tu maîtrises le concept, pas seulement l'outil Kafka.

Réponse — la théorie d'abord, brièvement : **[THÉORIE]** > Dans une architecture classique, un service en appelle un autre directement : il connaît son adresse, attend sa réponse, et échoue si l'autre est indisponible. C'est un couplage fort et synchrone. > > Dans une architecture événementielle, un service **publie un fait accompli** — « une commande a été créée » — sans savoir qui l'écoute ni si quelqu'un écoute. D'autres services s'abonnent et réagissent. L'émetteur et le récepteur ne se connaissent pas ; ils ne partagent qu'un contrat d'événement.

Les quatre propriétés qui en découlent :

Propriété	Ce que ça change
Découplage	On ajoute un consommateur sans toucher au producteur
Asynchronisme	Le producteur n'attend pas ; il continue immédiatement
Tolérance aux pannes	Si le consommateur est indisponible, l'événement reste dans Kafka
Extensibilité	Plusieurs consommateurs traitent le même événement différemment

Dans notre projet [CODE]** — et c'est là que ça devient spécifique :**

L'architecture événementielle n'est pas un choix de style : **c'est ce qui rend le serverless complet**. Un consommateur Kafka classique doit tourner en permanence pour maintenir sa session de groupe — il ne peut jamais descendre à zéro. Chez nous, c'est la **KafkaSource** qui maintient la connexion Kafka ; l'application cliente reçoit l'événement en HTTP et peut rester à zéro instance entre deux événements.

Sans ce modèle, une application événementielle serait facturée en permanence, et le scale-to-zero perdrait tout son sens pour cette catégorie de charges.

Relance : « Donnez-moi un exemple concret dans votre projet. »

Utilise tes deux vrais microservices [CODE] :

`order-service` publie un événement de type `order.created` sur le topic `orders`. `notification-service` s'y abonne. Le point remarquable : `notification-service` a **une seule dépendance — Express**. Aucune bibliothèque Kafka. Il reçoit l'événement comme un simple `POST / HTTP`, avec les attributs dans les en-têtes `ce-type`, `ce-source`, `ce-id`. C'est la `KafkaSource` qui a fait toute la traduction.

Q-A6 — [EXPERT] Où vit le type d'un événement ? Qui le déclare ?

Niveau : [EXPERT] · **Domaine :** Event-driven

Ce que le jury vérifie : que tu comprends ton propre modèle, pas seulement que tu l'as fait marcher.

Réponse — la distinction est essentielle :

Le type n'est **pas** une ressource qu'on déclare quelque part. C'est un champ que le **producteur écrit dans chaque message** qu'il publie.

Côté producteur [CODE]** —** dans `order-service/index.js` :

```
const cloudEvent = {
  specversion: '1.0',
  type:        'order.created',      // ← le type naît ICI
  source:      'nextstep/order-service',
  ...
};
```

C'est écrit en dur dans le code du microservice. La plateforme ne l'apprend qu'à la première publication.

Côté consommateur [CODE]** —** le type est déclaré **dans la plateforme**, au déploiement, comme filtre du Trigger :

```
"filter", Map.of("attributes", Map.of("type", eventType))
```

Le tableau à retenir :

Rôle du service	Où vit le type	Qui l'écrit
Producteur	Dans son code source	Le développeur du microservice
Consommateur	Dans le filtre du Trigger	L'utilisateur, via le portail

La conséquence, et c'est un vrai piège : la correspondance est **exacte** (`filterType("exact")`). Si le producteur publie `order-created` et que le filtre attend `order.created`, il ne se passe **rien** — aucune erreur, aucun log d'échec. L'événement atteint le Broker, ne correspond à aucun Trigger, et disparaît. Le consommateur n'est jamais réveillé.

Comment retrouver un type oublié : les journaux. `EventService` écrit à chaque publication une entrée `EVENT_PUBLISHED` contenant `topic`, `type` et `id`. [CODE]

Limite à assumer si on te pousse : il n'existe **aucun endpoint** listant les types d'événements disponibles — le champ se saisit à la main, sans autocomplétion ni validation. **[NON IMPLÉMENTÉ]** Un catalogue de types alimenté depuis les publications observées supprimerait toute une classe d'erreurs. C'est une bonne perspective à proposer.

Q-A7 — Comment un Trigger est-il relié à son Broker ?

Niveau : [AVANCÉ] · **Domaine :** Event-driven

Réponse [CODE]** :** Par un **simple nom**, et c'est ce qui explique une contrainte importante.

```
"spec", Map.of(
  "broker", "default",      // ← le lien : un nom nu
  "filter", ...,
  "subscriber", Map.of("uri", subscriberUrl)
)
```

`spec.broker` n'est pas une URL ni une référence qualifiée : c'est le nom du Broker. Knative le cherche donc **dans le namespace du Trigger**. D'où l'exigence que Trigger et Broker

partagent le même namespace — et d'où l'appel à `ensureBrokerExists(namespace)` avant toute création de `KafkaSource`.

Les trois ressources, dans le namespace du tenant :

Ressource	Nom	Ce qui la relie
Broker	<code>default</code>	—
KafkaSource	<code><app>-source</code>	son <code>sink</code> pointe vers le Broker
Trigger	<code><app>-source-trigger</code>	<code>spec.broker: default</code> + <code>subscriber.uri</code> vers l'app

Le détail de robustesse à citer : `createTrigger()` est `@Transactional`. Si la création sur le cluster échoue, la transaction annule la ligne en base — on ne se retrouve jamais avec un Trigger qui « existe » en base mais pas sur le cluster. **[CODE]**

Le champ à connaître pour diagnostiquer : `GET /api/eventing/triggers` retourne `ready`. Un Trigger créé mais `ready: false` signale un problème — typiquement le Broker absent du namespace.

Q-A8 — [PIÈGE] Vos microservices passent-ils par votre API pour publier ?

Niveau : [PIÈGE] · **Domaine :** Architecture

Ce que le jury vérifie : connais-tu vraiment le comportement de ton propre code ?

Réponse honnête [CODE]** : Les deux chemins existent, et c'est important de le savoir.

Chemin 1 — via l'API de la plateforme. `POST /api/events` accepte un JWT ou une clé d'API, et `EventService` publie sur Kafka en ajoutant les en-têtes `CloudEvent`. Ce chemin est journalisé (`EVENT_PUBLISHED`), donc traçable et facturable.

Chemin 2 — en direct. `order-service` **contourne l'API** : il utilise `kafkajs` et se connecte directement au bootstrap Kafka du cluster.

```
brokers: ['my-cluster-kafka-bootstrap.kafka.svc.cluster.local:9092']
```

Comment le défendre : c'est cohérent pour une application cliente autonome — elle n'a pas à dépendre de notre API pour fonctionner, et cela évite un saut réseau supplémentaire. La `NetworkPolicy` autorise d'ailleurs explicitement l'egress vers le namespace `kafka`, donc c'est un usage prévu.

La limite à reconnaître : ces publications directes **n'apparaissent pas** dans les journaux `EVENT_PUBLISHED` de la plateforme. On perd la traçabilité et la possibilité de facturer à l'événement. Si l'on voulait imposer le passage par l'API, il faudrait retirer l'accès direct à Kafka de la `NetworkPolicy` — mais cela casserait le cas d'usage d'un consommateur Kafka classique.

Q-A9 — Pourquoi Kafka et pas RabbitMQ ?

Niveau : [INTERMÉDIAIRE] · Domaine : Event-driven

Réponse :

Trois raisons, dans l'ordre d'importance pour notre projet.

L'intégration native avec Knative Eventing. Il existe une KafkaSource officielle et maintenue. C'est le critère décisif : sans elle, il aurait fallu écrire nous-mêmes le pont entre le bus et le Broker.

Le modèle de log persistant. Kafka conserve les messages selon une politique de rétention, et chaque consommateur suit son propre offset. Un consommateur peut donc relire l'historique. RabbitMQ est une file : le message consommé disparaît.

Strimzi. L'opérateur Kubernetes de référence pour Kafka, ce qui rend l'exploitation déclarative et cohérente avec le reste de l'architecture.

Quand RabbitMQ serait meilleur : pour du routage complexe par clés, des files de tâches avec accusé de réception par message, ou une latence très faible sur de petits volumes. Ce n'était pas notre besoin.

Q-A10 — Comment votre plateforme se compare-t-elle à Cloud Run aujourd'hui ?

Niveau : [EXPERT] · Domaine : Recul

Ce que le jury cherche : de l'honnêteté, pas de la vantardise.

Réponse en trois temps :

Là où nous sommes comparables — le modèle d'exécution est le même, et pour cause : Cloud Run est lui-même bâti sur Knative. Déploiement d'une image, scale-to-zero, révisions, routage par URL générée.

Là où nous apportons quelque chose qu'ils n'ont pas — l'hébergement sur infrastructure privée. C'est le point qui justifie le projet pour NextStep IT : aucune offre d'hyperscaler ne peut tourner sur le cluster de l'intégrateur.

Là où nous sommes très en retrait, et il faut le dire — la haute disponibilité (nous avons une réplique de chaque composant), la sécurité de la chaîne d'approvisionnement (pas de signature ni de scan d'images), les tests automatisés (aucun), et l'échelle éprouvée. Cloud Run est un produit exploité par des centaines d'ingénieurs depuis des années ; le nôtre est un prototype fonctionnel de PFE.

La formule qui conclut bien : > Nous ne prétendons pas concurrencer Cloud Run. Nous démontrons qu'on peut construire, sur des briques open source, une plateforme au modèle équivalent, hébergeable là où Cloud Run ne peut pas aller.

Partie II — Questions par sprint

Sprint 0 — Mise en place de l'infrastructure Kubernetes

| 1. Compréhension du sprint

Ce sprint construit le socle : cluster à 3 nœuds, CNI Cilium pour le réseau et l'isolation, MetalLB pour les IP externes, Kourier comme passerelle d'entrée. Aucune fonctionnalité visible pour un utilisateur.

| Q0.1 — Pourquoi Kubernetes plutôt que Docker Compose ?

Niveau : [BASE] · **Domaine :** Architecture

Ce que le jury vérifie : que tu n'as pas choisi Kubernetes parce que « c'est moderne ».

Réponse courte à l'oral : > Docker Compose orchestre des conteneurs sur une seule machine, sans redémarrage automatique en cas de panne de nœud, sans mise à l'échelle, et surtout sans modèle d'isolation multi-tenant. Notre produit est une plateforme qui héberge les applications de plusieurs clients : il nous fallait des namespaces, des NetworkPolicy, des quotas et un ordonnanceur. Compose ne fournit aucun de ces mécanismes.

Réponse approfondie : Kubernetes apporte quatre choses que le projet exige structurellement — le placement automatique des charges sur un ensemble de machines, la réconciliation permanente entre état désiré et état réel, le cloisonnement par namespace, et un écosystème d'extensions (CRD) sans lequel ni Knative ni Strimzi n'existeraient.

Dans notre projet : notre backend ne parle jamais à Docker. Il crée des ressources Kubernetes via Fabric8 : `Namespace`, `NetworkPolicy`, `ResourceQuota`, et surtout des CRD Knative (`serving.knative.dev/v1`). **[CODE]** `KnativeService.java`, `QuotaService.java`.

Pourquoi pas une autre solution ? Docker Swarm est en fin de vie et n'a pas d'écosystème CRD. Nomad aurait pu convenir mais sans Knative ni Strimzi, il aurait fallu réimplémenter le scale-to-zero et l'opérateur Kafka.

Relance probable : « Kubernetes vous apporte donc la haute disponibilité ? »

Réponse à la relance : Non, pas en l'état. Kubernetes nous donne les *mécanismes* de la haute disponibilité, mais notre déploiement ne l'exploite pas : `replicas: 1` sur le backend **[CODE]** `k8s/backend/deployment.yaml`, un seul control-plane, un seul broker Kafka, une seule instance PostgreSQL. Nous avons la plateforme pour la HA, pas la configuration.

Erreur à éviter : dire que le projet est hautement disponible. Ta propre configuration te contredit.

| Q0.2 — Expliquez le rôle de chaque composant du control-plane

Niveau : [INTERMÉDIAIRE] · **Domaine :** Kubernetes

Réponse courte : > L'**API Server** est le point d'entrée unique : tout passe par lui, y compris notre backend. **etcd** stocke l'état désiré du cluster. Le **Scheduler** décide sur quel nœud placer un Pod. Le **Controller Manager** fait converger l'état réel vers l'état désiré. Sur chaque nœud, le **kubelet** démarre et surveille les conteneurs.

Dans notre projet : notre backend est un **client de l'API Server**, authentifié par le ServiceAccount monté dans son pod. **[CODE]** Quand `KnativeService.deploy()` crée un `Service` Knative, il écrit dans etcd via l'API Server ; c'est ensuite le controller de Knative Serving qui crée la Revision, puis le Deployment, puis le Scheduler qui place le Pod sur `vm02` ou `vm03`.

Relance : « Que se passe-t-il si le control-plane tombe ? »

Réponse à la relance : Les applications déjà déployées **continuent de tourner** — les kubelets et les conteneurs sont indépendants du control-plane. En revanche, plus rien de nouveau n'est possible : notre backend ne peut plus déployer, la mise à l'échelle Knative s'arrête, et une application à zéro instance ne se réveillera plus. Avec un seul control-plane, c'est notre **principal point de défaillance unique côté infrastructure**.

| Q0.3 — Pourquoi Cilium, et pas Calico ou Flannel ?

Niveau : [AVANCÉ] · **Domaine :** Réseau

Réponse courte : > Flannel est éliminé d'emblée : il ne supporte pas nativement les NetworkPolicy, or l'isolation entre clients est une exigence centrale, pas une option. Restaient Calico et Cilium, comparables sur ce point. Nous avons retenu Cilium pour son architecture eBPF, qui filtre dans le noyau sans passer par iptables, et pour son potentiel d'observabilité réseau.

[THÉORIE] Un CNI (Container Network Interface) attribue une IP à chaque Pod et achemine le trafic. Kubernetes ne l'implémente pas lui-même.

Dans notre projet : Cilium 1.19.1, vérifié par capture `cilium status`. Il applique les `NetworkPolicy` créées par notre backend. **[CODE]**

Erreur à éviter : prétendre avoir mesuré des performances comparées. Assume : « les deux répondaient au besoin, j'ai choisi Cilium pour son évolutivité ».

| Q0.4 — [PIÈGE] Un namespace garantit-il l'isolation entre deux tenants ?

Niveau : [PIÈGE] · **Domaine :** Multi-tenance

Ce que le jury vérifie : si tu confonds cloisonnement logique et isolation réelle.

Réponse : > Non, et c'est un point important. Un namespace isole les **noms** : deux ressources peuvent porter le même nom dans deux namespaces différents. Il ne bloque **rien** au niveau réseau — par défaut, tous les pods du cluster peuvent se joindre, quel que soit leur namespace.

Ce qu'un namespace n'isole PAS : le réseau (sans NetworkPolicy), les ressources CPU/mémoire (sans ResourceQuota), le noyau Linux (les conteneurs le partagent), les nœuds (les pods de deux tenants tournent sur la même machine).

Dans notre projet, l'isolation repose sur trois couches : [CODE] 1. Namespace `user-
<username>`, créé dynamiquement — `UserContextService.buildNamespace()` 2. NetworkPolicy default-deny appliquée par Cilium — `KnativeService.ensureNetworkPolicyExists()` 3. Vérification de propriété applicative — `AppService.requireOwned()`

Limite honnête : l'isolation est logicielle, pas matérielle. Un tenant reste sur le même noyau que les autres. Une isolation forte demanderait des runtimes de type gVisor ou Kata Containers, ou des nœuds dédiés.

Q0.5 — Pourquoi MetalLB ? Et comment une requête externe atteint-elle un Pod ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Réseau

Réponse courte : > Sur un cloud public, un Service de type LoadBalancer reçoit automatiquement une IP publique du fournisseur. Sur notre cluster bare-metal, ce contrôleur n'existe pas : le Service resterait indéfiniment en `<pending>`. MetalLB joue ce rôle en attribuant des IP depuis une plage réservée du réseau local.

Le trajet complet [CODE]**.*

```
Client → résolution <app>.<ns>.<ip>.sslip.io
        → IP externe attribuée par MetalLB
        → Kourier (lit le nom d'hôte)
        → Route Knative → Revision
        → [si 0 instance] Activator → Autoscaler → Pod démarre
        → Pod (queue-proxy + conteneur client)
```

Détail vérifié : le portail `platform-web` est exposé en LoadBalancer sur `10.9.21.238`, la console `platform-admin` reste volontairement en NodePort 30081. [CODE] `application-k8s.yml` (liste CORS).

Relance : « Différence entre Service, Ingress et LoadBalancer ? »

Réponse : Un **Service** donne une adresse stable à un groupe de pods, à l'intérieur du cluster (ClusterIP). Un **LoadBalancer** est un type de Service qui demande une IP externe. Un **Ingress** (ici Kourier) route au niveau HTTP selon le nom d'hôte ou le chemin, ce qu'un Service ne sait pas faire. Chez nous les trois coexistent : MetalLB fournit l'IP, Kourier route par nom d'hôte, le Service atteint le pod.

Q0.6 — [EXPERT] Votre convention `sslip.io` est-elle viable en production ?

Niveau : [EXPERT] · **Domaine :** Réseau

Réponse : > Non, et c'est assumé. `sslip.io` est un service public qui résout tout nom contenant une IP vers cette IP. C'est parfait pour un cluster de développement sans

infrastructure DNS : zéro configuration. En production, il faudrait un domaine dédié, une zone DNS, un certificat TLS wildcard, et idéalement `external-dns` pour créer les enregistrements automatiquement.

Coût actuel : URL peu lisibles, dépendance à un service tiers pour la résolution, et pas de HTTPS.

Sprint 1 — Déploiement de Knative Serving

1. Compréhension du sprint

Installation de Knative Serving, raccordement à Kourier, et capacité du backend à créer des ressources Knative par programmation via Fabric8.

Q1.1 — Pourquoi Knative plutôt que des Deployments Kubernetes classiques ?

Niveau : [BASE] · **Domaine :** Cloud Native

Réponse courte : > Trois raisons. Le **scale-to-zero** : Kubernetes ne sait pas descendre un Deployment à zéro réplica puis le réveiller sur une requête HTTP — il faudrait réimplémenter l'Activator, la file d'attente et le réveil. Le **modèle de révisions** : chaque modification crée une Revision figée, ce qui donne l'historique et le rollback gratuitement. Le **routing** : Knative génère l'URL et la Route, sans écrire un Ingress par application.

Dans notre projet : `KnativeService.deploy()` ne crée **qu'une seule ressource**, un `Service` de l'API `serving.knative.dev/v1`. Revision, Deployment et Pod en sont dérivés automatiquement. **[CODE]**

Erreur à éviter : dire que tu as « codé le scale-to-zero ». C'est natif à Knative, et c'est une bonne décision de ne pas l'avoir réimplémenté. Dis-le ainsi.

Q1.2 — [EXPERT] Tracez le cycle complet d'un scale-to-zero et d'un réveil

Niveau : [EXPERT] · **Domaine :** Knative

Réponse (9 étapes) : [THÉORIE + MÉMOIRE]

1. État de repos : 0 pod. La Route pointe vers l'**Activator**, pas vers l'application.
2. Une requête HTTP arrive sur **Kourier**.
3. Kourier la transmet à l'Activator, qui **met la requête en file d'attente**.
4. L'Activator signale la demande à l'**Autoscaler (KPA)**.
5. L'Autoscaler passe le Deployment de la Revision de 0 à 1 réplica.
6. Le Pod démarre : conteneur client + sidecar **queue-proxy**.
7. L'Activator transmet la requête en attente au Pod.
8. Le queue-proxy remonte en continu les métriques de concurrence à l'Autoscaler.
9. Après inactivité prolongée, l'Autoscaler redescend à 0.

Le détail qui prouve la maîtrise : le **queue-proxy**. C'est lui qui mesure la concurrence réelle et permet à l'Autoscaler de décider. Cite-le.

Limite honnête : cette séquence n'a pas été capturée expérimentalement dans le mémoire — elle exigeait une observation dans le temps. **[MÉMOIRE]**

Relance : « Le cold start, c'est un problème ? »

Réponse : Oui, c'est le compromis du scale-to-zero. La première requête après inactivité attend le démarrage du pod — pour une application Spring Boot, plusieurs secondes. Trois parades existent : fixer `minScale: 1` pour les applications critiques (notre valeur par défaut est 0), réduire la taille de l'image, ou utiliser une compilation native. Aucune n'est appliquée par défaut chez nous, c'est un choix du client via le formulaire de déploiement.

Q1.3 — [AVANCÉ] Pourquoi Fabric8 plutôt que d'exécuter

`kubectl` ?

Niveau : [AVANCÉ] · **Domaine** : Backend

Réponse courte : > Exécuter `kubectl` depuis Java voudrait dire lancer un processus externe, construire des chaînes de commande, et parser du texte pour récupérer un résultat. Fabric8 est un client Java typé de l'API Kubernetes : il gère l'authentification, la sérialisation, les erreurs sous forme d'exceptions, et surtout les **watch**, impossibles avec `kubectl` dans un serveur.

Dans notre projet [CODE]** : ** `io.fabric8:kubernetes-client:6.10.0` dans le `pom.xml`. Le client détecte automatiquement le jeton du ServiceAccount monté dans le pod. Il sert à : - créer les `Service Knative` (`genericKubernetesResources("serving.knative.dev/v1", "Service")`) - créer namespaces, NetworkPolicy, ResourceQuota - créer Broker, Trigger, KafkaSource - lire les logs de pods, les nœuds, les PVC - ouvrir un **watch permanent** — `KnativeWatcher`

Le point technique fort : il n'existe **pas de client Java officiel pour Knative**. Nous passons donc par l'API générique de Fabric8, en construisant les manifestes comme des structures non typées — plus verbeux et sans vérification à la compilation. C'est un compromis assumé, appliqué à toutes les CRD du projet pour rester homogène.

Relance : « Comment votre backend s'authentifie-t-il auprès de l'API Server ? »

Réponse : Par le ServiceAccount monté dans le pod ; Fabric8 lit automatiquement le jeton et le certificat de l'autorité depuis le système de fichiers. **Point d'honnêteté à connaître** : nous utilisons le ServiceAccount `default` du namespace `platform`, pas un ServiceAccount dédié — c'est documenté dans notre `rbac.yaml` comme un manque. **[CODE]**

Q1.4 — [EXPERT] Que fait exactement le `KnativeWatcher` ?

Niveau : [EXPERT] · **Domaine** : Backend

Réponse : > C'est le composant qui maintient la cohérence entre l'état du cluster et notre base de données. Au démarrage, il ouvre un **watch permanent** sur les ressources Knative de tous les namespaces. À chaque événement du cluster, il met à jour le statut de l'application en base. Il gère sa propre reconnexion en cas de coupure.

Le mécanisme complémentaire : `AppService.syncStatusFromKubernetes()` interroge le cluster à chaque consultation de liste ou de détail, compare avec la base, et n'écrit que si

quelque chose a changé. La correspondance est explicite : `RUNNING` → `RUNNING` , `SCALED_TO_ZERO` → `IDLE` , `NOT_FOUND` → `DELETED` . **[CODE]**

Pourquoi les deux ? Le watcher couvre les changements survenant quand personne ne regarde. La synchronisation à la lecture couvre le cas où le watcher aurait manqué un événement, typiquement après un redémarrage du backend. C'est une redondance volontaire.

Relance : « Votre backend est-il un opérateur Kubernetes ? »

Réponse : Non. Un opérateur réconcilie en boucle l'état réel vers l'état désiré. Le `KnativeWatcher` **observe** et met à jour la base, il ne modifie jamais le cluster. Notre backend est un client de l'API, pas un opérateur.

Sprint 2 — Mise en place de Kafka et Strimzi

Q2.1 — Expliquez les notions fondamentales de Kafka

Niveau : [BASE] · Domaine : Kafka

Réponse courte : [THÉORIE] > Un **broker** stocke les messages. Un **topic** est un flux nommé, subdivisé en **partitions** — l'unité de parallélisme et d'ordonnancement. Un **producer** publie, un **consumer** lit. Un **consumer group** répartit les partitions entre ses membres. L'**offset** est la position de lecture, conservée par le groupe.

Point de nuance à connaître : Kafka ne garantit l'ordre **qu'à l'intérieur d'une partition**, jamais à l'échelle du topic. C'est une question classique.

Q2.2 — [AVANCÉ] Montrez-moi où Kafka est configuré dans votre code

Niveau : [AVANCÉ] · Domaine : Kafka

Réponse [CODE] : **** À trois endroits, et la différence entre eux est instructive :

Fichier	<code>bootstrap.servers</code>	Contexte
<code>application-k8s.yml</code>	<code>my-cluster-kafka-bootstrap.kafka.svc.cluster.local:9092</code>	Backend dans le cluster — DNS interne
<code>application.yml</code> / <code>application-local.yml</code>	<code>10.9.21.233:9093</code>	Développement local — écouteur externe

Pourquoi deux adresses différentes ? C'est une excellente question de relance. Le nom `my-cluster-kafka-bootstrap.kafka.svc.cluster.local` n'est résolvable **que depuis l'intérieur du cluster** — c'est du DNS Kubernetes. Un backend lancé sur un poste de développement ne peut pas le résoudre ; il passe donc par un écouteur externe de type LoadBalancer sur le port 9093.

Le nom DNS se décompose ainsi : `<service>.<namespace>.svc.cluster.local`.

Côté code : `KafkaService` crée un `AdminClient` avec `AdminClientConfig.BOOTSTRAP_SERVERS_CONFIG`, un timeout de 5 s, et l'utilise pour `createTopics()`, `listOffsets()` et la lecture du décalage des groupes de consommateurs. **[CODE]**

Q2.3 — [AVANCÉ] Pourquoi AdminClient et pas Fabric8, alors que Strimzi expose une CRD `KafkaTopic` ?

Niveau : [AVANCÉ] · Domaine : Kafka / Architecture

Réponse : > Parce que ce sont deux plans différents. Fabric8 parle à l'**API Kubernetes**. Créer un topic est une opération du **protocole Kafka lui-même**, qui s'adresse aux brokers, pas à l'API Server.

Strimzi propose bien une CRD `KafkaTopic`, mais elle ajoute une indirection : on crée un objet Kubernetes, l'opérateur le voit, et crée le vrai topic — de façon asynchrone. Avec `AdminClient`, la création est **synchrone** et l'erreur remonte immédiatement, ce qui permet de répondre correctement au client.

!/ Piège de nommage à devancer : il existe une classe `KafkaTopic` dans notre backend. Ce n'est **pas** la CRD Strimzi : c'est une entité JPA qui persiste les métadonnées (propriétaire, nom, partitions). Si le jury voit ce nom, devance la confusion.

Nuance honnête : notre `rbac.yaml` accorde bien les droits sur `kafka.strimzi.io/kafkatopics` **[CODE]**, ce qui suggère qu'un usage de la CRD a été envisagé. Le code retenu passe par `AdminClient`.

| Q2.4 — [PIÈGE] Kafka garantit qu'aucun message n'est perdu ?

Niveau : [PIÈGE] · **Domaine :** Kafka

Réponse : > Kafka *peut* garantir la durabilité, mais **pas dans notre configuration**. La garantie repose sur la réplication : un facteur de réplication de 3 avec `min.insync.replicas=2` permet de survivre à la perte d'un broker.

Notre cluster déclare **un seul broker** **[CODE]**. Le facteur de réplication ne peut donc pas dépasser 1 : si le broker est perdu, les messages non consommés le sont aussi.

C'est un dimensionnement d'environnement de développement, suffisant pour valider la chaîne fonctionnelle. En production, il faudrait au minimum trois brokers.

Bonus à savoir : les sémantiques de livraison sont *at-most-once*, *at-least-once* (le défaut), et *exactly-once* (via les transactions Kafka). Nous n'avons rien configuré de spécifique, donc *at-least-once*.

| Q2.5 — Pourquoi Strimzi ? Qu'est-ce qu'un opérateur ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Kafka / Kubernetes

Réponse : > Un **opérateur** est un contrôleur qui étend Kubernetes avec des CRD et de la logique métier : il sait non seulement créer des ressources, mais aussi les exploiter — mises à jour, reprise sur panne, reconfiguration. C'est la façon d'encoder l'expertise d'un administrateur dans du logiciel.

Dans notre projet : Strimzi 0.45.0 opère le cluster `my-cluster` dans le namespace `kafka`. Il crée les brokers, ZooKeeper, et génère automatiquement le service de bootstrap. **[CODE]**

Alternative rejetée : installer Kafka manuellement sur des VM. Cela aurait signifié gérer soi-même la configuration, les certificats, les mises à jour et la reprise sur panne — hors du périmètre d'un PFE, et contraire à l'approche Cloud Native du projet.

[PIÈGE] Piège possible : « Vous utilisez ZooKeeper ou KRaft ? »**

Réponse honnête : **ZooKeeper [CODE]**. KRaft est le mode moderne qui supprime cette dépendance, et Strimzi 0.45 le supporte — ce n'était donc pas une contrainte technique, mais le mode par défaut conservé sans arbitrage explicite. À refaire, KRaft serait le bon choix.

Sprint 3 — Gestion des utilisateurs et des accès

Q3.1 — [EXPERT] Quel flow OAuth2 utilisez-vous exactement ?

Niveau : [EXPERT] [PIÈGE] · Domaine : Sécurité

C'est LA question la plus dangereuse de ta soutenance. Prépare-la en priorité absolue.

Réponse : > Le portail utilise le flow **Resource Owner Password Credentials (ROPC)** : il envoie directement le login et le mot de passe à l'endpoint token de Keycloak avec `grant_type=password`, et reçoit le jeton en retour. **[CODE]** `AuthContext.jsx`.

Assume immédiatement : ce n'est pas le flow recommandé. ROPC est déprécié dans OAuth 2.1 pour une raison précise : l'application cliente manipule le mot de passe en clair. C'est acceptable uniquement quand client et serveur d'autorisation appartiennent à la même organisation — notre cas — mais cela empêche structurellement le SSO, la fédération d'identité et le MFA porté par Keycloak.

Le flow correct : Authorization Code + PKCE — redirection vers la page de login Keycloak, retour avec un code, échange du code contre un jeton. Le mot de passe ne transite jamais par l'application.

Détail à connaître avant que le jury ne le trouve : un fichier `web-portal/src/auth/keycloak.js` utilisant le vrai adaptateur `keycloak-js` existe dans le dépôt mais **n'est importé nulle part** — c'est du code mort, vestige d'une première approche. **[CODE]**

Stratégie en trois temps : 1. Nomme le flow correctement, sans hésiter : ROPC. 2. Explique pourquoi il est déconseillé, en montrant que tu connais PKCE. 3. Explique le contexte qui l'a rendu acceptable, et cite la migration comme perspective identifiée.

Erreur fatale : dire « j'utilise OAuth2 avec Keycloak » en espérant que ça passe. Si le jury creuse et découvre que tu ne savais pas quel flow tu utilisais, c'est bien plus grave que le choix lui-même.

Q3.2 — Comment un JWT est-il validé ? Keycloak est-il appelé à chaque requête ?

Niveau : [INTERMÉDIAIRE] · Domaine : Sécurité

Réponse courte : > Non, et c'est tout l'intérêt d'un JWT. Un jeton contient trois parties : un **header** (algorithmes), un **payload** (les claims : `sub`, `iss`, `exp`, `preferred_username`, rôles) et une **signature**. Le backend vérifie la signature avec la clé publique de Keycloak, récupérée une fois via l'URL de découverte du realm, puis mise en cache. Aucun appel réseau par requête.

Dans notre projet [CODE]** : ** SecurityConfig configure oauth2ResourceServer avec un KeycloakJwtAuthConverter maison qui extrait le rôle applicatif du jeton. La session est en mode STATELESS : le backend ne stocke rien.

issuer-uri vaut http://keycloak.platform.svc.cluster.local:8080/realms/platform en cluster, http://localhost:8081/realms/platform en local. [CODE]

Pourquoi la falsification est impossible : un attaquant peut lire un JWT (il est seulement encodé en base64, **pas chiffré**), mais il ne peut pas le modifier sans invalider la signature, dont il n'a pas la clé privée.

Relance : « Que se passe-t-il si le token expire ? »

Réponse : Le backend renvoie 401. Côté portail, un mécanisme de rafraîchissement automatique utilise le refresh_token pour obtenir un nouveau jeton sans réauthentification. [CODE] AuthContext.jsx .

Relance piège : « Pourquoi ne pas sécuriser uniquement le frontend ? »

Réponse : Parce que le frontend n'est pas une frontière de sécurité. Cacher un bouton dans React n'empêche personne d'appeler l'API directement avec curl. Toute la sécurité est côté backend ; le frontend ne fait que masquer ce qui serait de toute façon refusé.

| Q3.3 — Différence entre rôle et permission dans votre modèle ?

Niveau : [INTERMÉDIAIRE] · **Domaine** : Sécurité

Réponse [CODE]** : > **Trois rôles**** : ADMIN (exploitant de la plateforme), CLIENT_ADMIN (client titulaire d'un compte), MEMBER (collaborateur invité). Et huit **permissions** granulaires, qui ne s'appliquent qu'aux MEMBER : DEPLOY_APP, DELETE_APP, MANAGE_KAFKA, MANAGE_EVENTING, VIEW_LOGS, VIEW_MONITORING, VIEW BILLING, EXPORT BILLING .

La règle, en une ligne de code :

```
if (!"MEMBER".equals(u.getRole())) return true; // ADMIN / CLIENT_ADMIN
return u.getPermissions() != null && u.getPermissions().contains(permission);
```

Comment c'est appelé : par annotation Spring, directement sur l'endpoint : @PreAuthorize("@permissionService.has(authentication.name, 'DEPLOY_APP')") [CODE] ApplicationController.createApp()

Relance quasi certaine : « @permissionService, c'est du Spring ? »

Réponse : Non, c'est **notre** bean. Le @ est une syntaxe SpEL qui résout un bean par son nom dans le contexte Spring. PermissionService étant annotée @Service sans nom explicite, Spring l'enregistre sous permissionService . Ce qui vient de Spring, c'est le mécanisme et la variable authentication ; la logique est la nôtre.

Prérequis à ne pas oublier : tout cela ne fonctionne que grâce à @EnableMethodSecurity sur SecurityConfig . Sans elle, les @PreAuthorize seraient silencieusement ignorés.

| Q3.4 — [EXPERT] Si je change l'ID dans l'URL, que se passe-t-il ?

Niveau : [EXPERT] · **Domaine** : Sécurité

Ce que le jury teste : l'IDOR (Insecure Direct Object Reference).

Réponse : > Le client A est authentifié, il a la permission `DELETE_APP`. Il appelle `DELETE /api/apps/{id-de-B}` avec son propre jeton, parfaitement valide. Ni l'authentification ni la vérification de permission ne le bloquent : il a le droit de supprimer des applications — simplement pas celle-là.

C'est le troisième étage de contrôle qui l'arrête :

```
private App requireOwned(String appId, String userId) {
    App app = appRepository.findById(appId)
        .orElseThrow(() -> new NotFoundException("App not found: " + appId));
    if (!app.getUserId().equals(userId)) {
        throw new UnauthorizedException("Access denied to app: " + appId);
    }
    return app;
}
```

Le détail qui compte : `requireOwned` est appelé dans **toutes** les méthodes recevant un `appId` externe — `getApp`, `updateApp`, `redploy`, `deleteApp`, `listRevisions`, `rollback`. **[CODE]** Un seul oubli rouvrirait la faille.

`listApps` ne l'appelle pas, mais filtre en amont par `findByUserId` : elle ne peut structurellement pas retourner l'application d'un autre.

Le `userId` passé n'est jamais le nom de l'utilisateur connecté — c'est `ctx.effectiveUserId()`, résolu par `UserService.resolve()`. Pour un `MEMBER`, c'est l'identifiant de son `CLIENT_ADMIN` : il accède donc aux applications de son équipe, jamais à celles d'une autre.

Code HTTP : `UnauthorizedException` est traduite en **403 Forbidden** par `GlobalExceptionHandler`, pas 401 — cohérent : 401 dirait « identifie-toi », or il est identifié. **[CODE]**

Nuance experte, si on te pousse : `requireOwned` renvoie 404 si l'application n'existe pas et 403 si elle appartient à un autre. Un attaquant peut donc **énumérer les identifiants valides**. Le durcissement consiste à renvoyer 404 dans les deux cas — ce que font GitHub et GitLab. C'est une fuite d'information mineure, pas un contournement du contrôle d'accès, mais savoir la nommer joue en ta faveur.

Q3.5 — Comment un `CLIENT_ADMIN` ajoute-t-il un membre ? Y a-t-il un e-mail d'invitation ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Métier / Sécurité

Réponse [CODE] :** > **Non, et c'est un choix à assumer. Le `CLIENT_ADMIN` saisit lui-même le nom d'utilisateur, l'e-mail et le mot de passe. `TeamService.addMember()` vérifie l'unicité, crée le compte dans Keycloak avec un mot de passe permanent** (pas de réinitialisation forcée), puis crée l'utilisateur local avec `role="MEMBER"` et `ownerId` pointant vers le `CLIENT_ADMIN`.**

Les quatre endpoints de `TeamController` sont protégés par `@PreAuthorize("hasRole('CLIENT_ADMIN')")` — un `ADMIN` n'y a **pas** accès.

Limites à reconnaître si on te pousse : - Un vrai flux d'invitation par e-mail avec jeton à usage unique serait plus sûr : le CLIENT_ADMIN ne connaîtrait pas le mot de passe de son collaborateur. - Aucun rôle de royaume Keycloak n'est assigné au membre : la délégation n'existe que dans notre base locale.

Q3.6 — Que se passe-t-il si quelqu'un s'inscrit via le formulaire public ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Sécurité

Réponse : > `POST /api/auth/register` est effectivement public [CODE] `SecurityConfig`. Le compte est créé dans Keycloak **sans rôle de royaume**, puis `UserSyncFilter` crée l'utilisateur local avec le rôle `MEMBER` par défaut et **sans `ownerId`**.

C'est volontairement inerte : ce compte ne peut rien faire. Comme il a le rôle `MEMBER`, `PermissionService.has()` le vérifie contre une liste de permissions vide. Toute action protégée est refusée tant qu'un ADMIN ne change pas son rôle via `PATCH /api/users/{id}/role`.

C'est une file d'attente de validation, pas une faille.

Q3.7 — Comment sont stockées les clés d'API ?

Niveau : [AVANCÉ] · **Domaine :** Sécurité

Réponse [CODE]** : > **La clé est générée avec `SecureRandom`, et** seule son empreinte SHA-256 est persistée**. La valeur en clair n'est retournée qu'une fois, à la création. Un préfixe de 12 caractères est stocké en clair, uniquement pour permettre à l'utilisateur d'identifier visuellement sa clé.

À la vérification, `ApiKeyFilter` recalcule le SHA-256 et cherche la correspondance parmi les clés actives.

Relance experte : « **Pourquoi SHA-256 et pas BCrypt, comme pour les mots de passe ?** »

Excellente question, et il y a une vraie réponse : BCrypt est **volontairement lent**, ce qui protège un mot de passe humain à faible entropie contre le bruteforce. Une clé générée par `SecureRandom` a une entropie très élevée : le bruteforce n'est pas la menace. Et elle est vérifiée à chaque requête d'un pipeline CI/CD — la lenteur de BCrypt serait un coût sans bénéfice. SHA-256 est le choix standard pour ce cas d'usage.

Détail intéressant : `ApiKeyController` n'a **aucune** annotation `@PreAuthorize` [CODE] — tout utilisateur authentifié, y compris un `MEMBER`, peut gérer ses propres clés.

Sprint 4 — Gestion des applications

Q4.1 — [EXPERT] Tracez tout ce qui se passe quand un CLIENT_ADMIN déploie une application

Niveau : [EXPERT] · Domaine : Architecture / Backend

C'est le cas d'utilisation central de ton projet. Tu dois le dérouler sans hésiter.

Réponse [CODE]**:**

Phase synchrone (réponse immédiate au client) 1. Le formulaire du portail envoie POST /api/apps avec image, tag, port, ressources, bornes d'échelle. 2. @PreAuthorize("@permissionService.has(authentication.name, 'DEPLOY_APP')") filtre l'appel. 3. AppService.createApp() appelle userContextService.resolve(username) → identifiant effectif et namespace du tenant. 4. Le nom du service Knative est généré (generateServiceName), l'entité App est persistée avec le statut DEPLOYING. 5. Un journal DEPLOYMENT_START est écrit et poussé en SSE. 6. deploymentAsyncRunner.triggerDeploy(app, req) est appelé, puis le backend répond 201 Created immédiatement.

Phase asynchrone (tâche de fond) 7. KnativeService.deploy() appelle ensureNamespaceExists(), qui crée le namespace et sa NetworkPolicy default-deny s'ils n'existent pas. 8. Le manifeste du Service Knative est construit et soumis à l'API Kubernetes. 9. En cas de conflit 409, le service est supprimé puis recréé. 10. L'URL est récupérée par sondage (jusqu'à 20 tentatives). 11. Statut mis à RUNNING, journal DEPLOYMENT_SUCCESS poussé en SSE. En cas d'échec : FAILED + DEPLOYMENT_FAIL. 12. Optionnellement, si le formulaire renseigne un topic Kafka, KafkaSource et Trigger sont créés dans la foulée (KAFKA_WIRED).

Puis, côté Knative : Service → Revision → Deployment → Pod.

Pourquoi asynchrone ? Parce que tirer l'image et démarrer le conteneur peut prendre des dizaines de secondes. Bloquer la requête HTTP donnerait une interface figée et risquerait un timeout.

Q4.2 — [EXPERT] Le piège du proxy Spring : pourquoi AppDeploymentAsyncRunner est-il une classe séparée ?

Niveau : [EXPERT] · Domaine : Backend / Spring

C'est ta meilleure preuve de maîtrise technique. Si tu ne prépares qu'une question sur le backend, prépare celle-ci.

Réponse : > Parce que @Async ne prend effet que sur les appels qui passent par le **proxy Spring** du bean où l'annotation est déclarée. Un appel intra-classe — AppService appelant

sa propre méthode `@Async` — contourne entièrement ce proxy et s'exécute **de façon synchrone**, sans aucune erreur ni avertissement.

Mettre la méthode sur un bean séparé garantit que chaque appelant y accède par un vrai appel inter-bean, donc via le proxy, donc que `@Async` s'applique réellement.

[CODE] Ce raisonnement est écrit en commentaire dans `AppDeploymentAsyncRunner.java` — tu peux inviter le jury à l'ouvrir.

Pourquoi c'est fort : c'est un piège classique de Spring qui provoque des bugs silencieux. L'avoir identifié, compris et contourné explicitement montre que tu maîtrises le framework, pas seulement son usage nominal.

Q4.3 — Comment fonctionne le retour à une révision antérieure ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Knative

Réponse [CODE]** : `> KnativeService.rollbackToRevision()` récupère la ressource Service Knative, puis patche son champ `spec.traffic` pour envoyer 100 % du trafic** vers la révision choisie :

```
spec.put("traffic", List.of(Map.of("revisionName", revisionName, "percent", 100)));
```

La liste des révisions vient de `listRevisions()`, qui interroge l'API avec le label `serving.knative.dev/service=<serviceName>` et trie par date de création décroissante.

Ce que ça montre : le rollback ne redéploie rien. Les révisions Knative sont immuables et coexistent ; on ne fait que rediriger le trafic. C'est instantané.

Ouverture : ce même mécanisme `spec.traffic` permettrait un déploiement canari — 90 % sur la révision stable, 10 % sur la nouvelle. **[NON IMPLÉMENTÉ]** chez nous, mais c'est la bonne réponse si on te demande une évolution.

Q4.4 — [AVANCÉ] Pourquoi une application supprimée reste-t-elle en base ?

Niveau : [AVANCÉ] · **Domaine :** Métier / Données

Réponse [CODE]** : `> AppService.deleteApp()` détruit réellement le service Knative sur le cluster, mais met le statut à `DELETED` en base au lieu de supprimer la ligne. Le commentaire du code l'explique : « *billing history must be preserved* ».

La logique est cohérente entre deux domaines : les instantanés de facturation antérieurs restent valides — ce qui était dû avant la suppression reste facturable — mais aucun coût nouveau ne s'accumule, car `BillingService.takeSnapshot()` filtre explicitement les applications `DELETED`.

Ce que ça prouve : un client ne peut pas effacer sa dette en supprimant son application, ni être facturé pour une ressource qui n'existe plus.

Q4.5 — Comment détectez-vous qu’une application cliente plante en boucle ?

Niveau : [AVANCÉ] · **Domaine :** Observabilité

Réponse [CODE]** : Un job planifié toutes les 5 minutes (`CrashLoopScheduler`, `@Scheduled(fixedRate = 5*60*1000)`) cherche les pods dont le compteur de redémarrages dépasse 5 — la signature d’un `CrashLoopBackOff`. Pour chaque application concernée, une alerte de type `CRASH_LOOP_ALERT` est créée et poussée en temps réel vers l’interface du client.

Un anti-spam évite la répétition : si une alerte du même type existe depuis moins d’une heure, elle n’est pas dupliquée (`CRASH_LOOP_ALERT_COOLDOWN_HOURS = 1`).

Ce que ça montre : tu as pensé à l’exploitation, pas seulement au chemin nominal.

Sprint 5 — Gestion de la messagerie événementielle

Q5.1 — [EXPERT] Un événement est publié. Tracez son chemin jusqu'à l'exécution du code client

Niveau : [EXPERT] · Domaine : Event-driven

C'est la question technique la plus riche de ton projet.

Réponse (11 étapes) :

1. Un producteur publie un message sur un **topic Kafka**.
2. La **KafkaSource** consomme ce message depuis le topic.
3. Elle l'encapsule au format **CloudEvent** et l'envoie en HTTP au Broker.
4. Le **Broker** Knative Eventing le reçoit.
5. Le **Trigger**, qui filtre sur le type d'événement, identifie l'application cible.
6. Si l'application est à zéro instance, l'**Activator** intercepte et met l'événement en file.
7. L'**Autoscaler (KPA)** est notifié.
8. Il passe le Deployment de 0 à 1 réplica.
9. Le **Pod** démarre, traite l'événement, répond HTTP 200.
10. Après inactivité, l'Autoscaler redescend à 0.
11. Le cycle recommence à la publication suivante.

La difficulté réelle rencontrée — à citer, elle vaut de l'or : Knative **exige qu'un Trigger et son Broker soient dans le même namespace**. Ce n'est pas évident à la lecture de la documentation, et le symptôme est vicieux : le Trigger reste silencieusement non fonctionnel, sans erreur. D'où l'appel à `ensureBrokerExists(namespace)` avant toute création de `KafkaSource`. **[CODE]** `EventingService.java`

Preuves d'exploitation : trois captures réelles dans le mémoire — `kubectl get broker`, `kubectl get trigger`, `kubectl get kafkasource`. **[CODE]**

Q5.2 — Pourquoi passer par un Broker ? Pourquoi pas de Kafka directement vers l'application ?

Niveau : [AVANCÉ] · Domaine : Event-driven

Réponse : > Parce que l'application cliente ne parle pas Kafka, elle parle **HTTP**. C'est tout l'intérêt du modèle : le client écrit un service HTTP ordinaire, sans bibliothèque Kafka, sans gestion d'offsets ni de consumer group. La `KafkaSource` fait la traduction Kafka → CloudEvent HTTP.

Le Broker apporte le découplage : plusieurs Triggers peuvent s'abonner au même Broker avec des filtres différents, et router le même flux vers des applications distinctes.

Et le scale-to-zero : un consommateur Kafka classique doit tourner en permanence pour maintenir sa session de groupe. Avec ce modèle, c'est la `KafkaSource` qui reste connectée ; l'application cliente peut descendre à zéro et n'être réveillée que quand un événement la concerne. **C'est précisément ce qui rend la plateforme serverless de bout en bout.**

| Q5.3 — Le câblage Kafka au déploiement, comment ça marche ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Event-driven

Réponse [CODE]** : Dans `AppDeploymentAsyncRunner.triggerDeploy()`, après un déploiement réussi :

```
if (Boolean.TRUE.equals(req.getKafkaEnabled()) && req.getKafkaTopicId() != null) {  
    var source = eventingService.createKafkaSource(...);  
    eventingService.createTrigger(app.getUserId(), source.getId(), filter, url);  
}
```

Le nom de la source est dérivé du service (`<serviceName>-source`), le filtre vaut `order.created` par défaut. L'opération est journalisée sous le type `KAFKA_WIRED`, donc traçable.

Valeur métier : le client coche une case dans le formulaire de déploiement et obtient une application événementielle complète, sans jamais toucher à Kubernetes ni à Kafka.

Sprint 6 — Gestion de l'observabilité applicative

| Q6.1 — Pourquoi SSE et pas WebSocket ou polling ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Observabilité

Réponse : > **SSE** (Server-Sent Events) est un canal HTTP **unidirectionnel** du serveur vers le navigateur. C'est exactement notre besoin : pousser des logs et des métriques. Le client n'a rien à renvoyer.

WebSocket serait bidirectionnel — une complexité inutile ici, avec un protocole distinct et une gestion manuelle de la reconnexion. SSE reconnecte automatiquement et passe par HTTP standard, donc par tous les proxys.

Le polling gaspille : requêtes répétées pour rien la plupart du temps, et latence égale à l'intervalle.

Dans notre projet [CODE]** : ** `LogSseService` maintient une `Map<userId, List<SseEmitter>>` — une liste, pour supporter plusieurs onglets ouverts. Les émetteurs sont créés avec `new SseEmitter(0L)` : **pas de timeout**.

Deux flux distincts : `GET /api/logs/stream` (journaux de déploiement) et `GET /api/logs/apps/{id}/pod-logs/stream` (logs bruts du conteneur, lus via l'API Kubernetes).

!/ Le point de sécurité à mentionner spontanément : l'objet `EventSource` du navigateur **ne permet pas d'envoyer un en-tête `Authorization`**. La première implémentation passait donc le jeton **dans les paramètres d'URL** — visible dans les logs d'accès, l'historique du navigateur et les en-têtes `Referer`. Cette vulnérabilité a été identifiée et corrigée : un filtre dédié, `SseTokenFilter`, gère désormais l'authentification de ces flux. [CODE]

Le mentionner toi-même est bien plus valorisant que de le laisser découvrir.

| Q6.2 — D'où viennent les métriques ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Observabilité

Réponse [CODE]** : > **De Prometheus****, interrogé en PromQL par `MetricsService` via un `WebClient` réactif. L'URL est configurée dans `application-k8s.yml` : `http://monitoring-stack-kube-prom-prometheus.monitoring.svc.cluster.local:9090`.

Le backend expose aussi ses propres métriques via `micrometer-registry-prometheus`, et l'actuator publie `health,info,prometheus,metrics`.

Un `PodMonitor` dédié au `queue-proxy` de Knative est versionné dans `k8s/monitoring/`, ce qui permet de récupérer les métriques de trafic par révision.

Relance : « Pourquoi Prometheus et pas des métriques maison ? »

Réponse : Parce que Prometheus est le standard de fait de l'écosystème Kubernetes : Knative, Strimzi et les composants système exposent déjà leurs métriques dans ce format.

Réimplémenter la collecte, le stockage temporel et le langage de requête aurait été un projet à part entière, sans valeur ajoutée.

Sprint 7 — Gestion de la facturation et des paiements

Q7.1 — Comment calculez-vous la facture d'un client ?

Niveau : [INTERMÉDIAIRE] · Domaine : Métier

Réponse [CODE]** : Par échantillonnage horaire. Chaque heure, `BillingScheduler` (`@Scheduled(cron = "0 0 * * * *")`) déclenche `takeSnapshot()`, qui parcourt toutes les applications non supprimées et enregistre un `BillingSnapshot` : CPU en vCPU, mémoire en Go, réplicas, et un facteur de disponibilité.

La formule :

```
coût_horaire = (vCPU × 0,048 + Go × 0,006) × réplicas × facteur
```

Le **facteur de disponibilité** traduit l'état réel :

Statut	Facteur	Sens
<code>RUNNING</code>	1,0	Application active
<code>FAILED</code>	0,0	Un déploiement raté n'est pas facturé
autres (<code>IDLE</code> , <code>DEPLOYING</code>)	0,2	Coût résiduel de réservation

Deux autres jobs : vérification quotidienne des impayés (`0 0 8 * * *`) et génération mensuelle des factures (`0 5 0 1 * *`).

Relance très probable : « Pourquoi facturer le CPU *demandé* et pas le *consommé* ? »

Réponse : Parce que Kubernetes **réserve** la ressource demandée sur le nœud, qu'elle soit utilisée ou non — elle est indisponible pour les autres tenants. C'est le modèle des fournisseurs cloud, et il incite le client à dimensionner justement. Facturer le consommé exigerait d'agréger des métriques Prometheus sur toute la période, avec plus de complexité et moins de précision.

Le 0,2 est-il défendable ? C'est une valeur de conception, pas une mesure. Elle représente le coût de disponibilité d'une application enregistrée. Assume-la comme un paramètre produit.

Q7.2 — [EXPERT] Comment savez-vous qu'un webhook Stripe vient réellement de Stripe ?

Niveau : [EXPERT] · Domaine : Sécurité / Paiement

Ce que le jury vérifie : l'endpoint webhook est public — comprends-tu le risque ?

Réponse [CODE]** : Par vérification de signature cryptographique. Stripe signe chaque webhook avec un secret partagé et place la signature dans l'en-tête `Stripe-`

Signature. Notre `PaymentController` récupère cet en-tête, et `PaymentService.handleWebhook()` appelle `Webhook.constructEvent(payload, sigHeader, webhookSecret)`.

Cette méthode de la bibliothèque officielle recalcule la signature à partir du corps brut et du secret : si elle ne correspond pas, elle lève une exception et l'événement est rejeté.

Le secret vient de `app.stripe.webhook-secret`, injecté par variable d'environnement — jamais en dur.

Pourquoi `/api/payment/webhook` est-il public dans `SecurityConfig` ? Parce que Stripe n'a pas de compte utilisateur sur notre plateforme : il ne peut pas présenter de JWT. La signature **remplace** l'authentification.

Relance experte : « Et l'idempotence ? »

Réponse honnête : Stripe peut renvoyer le même événement plusieurs fois — c'est documenté et normal. Nous n'avons **pas implémenté** de table d'événements traités pour dédupliquer sur l'`event.id`. **[NON IMPLÉMENTÉ]** C'est une limite réelle : un même événement rejoué pourrait être traité deux fois. La correction est simple — persister les identifiants d'événements traités et ignorer les doublons.

Sprint 8 — Comptes clients, quotas et audit

Q8.1 — Comment empêchez-vous un client de consommer tout le cluster ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Multi-tenance

Réponse [CODE]** : Par deux barrières complémentaires.

Barrière applicative : chaque tenant a un `TenantQuota` — par défaut 2000m CPU, 4 Gio, 10 applications. Avant toute création, `assertCanCreateApp()` compte les applications actives et lève une `ConflictException` (409) si le plafond est atteint, avec un message explicite plutôt qu'une 500 opaque.

Barrière Kubernetes : le quota est synchronisé vers le cluster sous forme de `ResourceQuota` nommée `tenant-quota` dans le namespace du tenant (`syncToCluster`). C'est important : cette seconde barrière est appliquée **par Kubernetes lui-même**. Même si le backend était contourné, le cluster refuserait le dépassement.

Détail d'audit à connaître : le droit RBAC sur `resourcequotas` **manquait initialement** — `syncToCluster()` échouait silencieusement (exception attrapée, journalisée, jamais appliquée). Le manque a été détecté et corrigé. [CODE] commentaire de `rbac.yaml`.

/! Contradiction à connaître : le mémoire mentionne par endroits « absence de limite de ressources par défaut ». Le code, lui, définit bien des valeurs par défaut. Si le jury relève l'écart, la vérité est celle du code.

Q8.2 — Que trace exactement votre journal d'audit ?

Niveau : [INTERMÉDIAIRE] · **Domaine :** Administration

Réponse [CODE]** : > **Uniquement les actions de gouvernance de l'ADMIN**, jamais les lectures**. Huit types exactement : `SUSPEND_CLIENT`, `RESTORE_CLIENT`, `SUSPEND_APP`, `RESTORE_APP`, `FORCE_DELETE_APP`, `FORCE_DELETE_TOPIC`, `UPDATE_QUOTA`, `SCALE_APP`.

Chaque entrée enregistre l'acteur, l'action, la cible, **l'état avant**, **l'état après**, et l'adresse IP source.

Défends le choix : un journal enregistrant chaque consultation deviendrait volumineux et illisible, noyant les événements qui comptent. L'audit répond ici à la question d'un litige ou d'un incident : « qui a changé quoi, et quel était l'état avant ? ».

Q8.3 — Techniquement, que fait la suspension d'un client ?

Niveau : [AVANCÉ] · **Domaine :** Administration

Réponse [CODE]** : > `KnativeService.scaleService()` **patche les annotations d'autoscaling du service Knative pour mettre `maxScale` à zéro**** : l'application ne peut plus démarrer d'instance et tout trafic reçoit un 503. Le statut passe à `SUSPENDED`.

La suspension d'un client entier applique ce traitement à toutes ses applications et marque l'utilisateur `suspended` . La restauration rétablit les bornes d'origine.

Pourquoi suspendre plutôt que supprimer : c'est **réversible et non destructif**. Les données, configurations et topics du client restent intacts. C'est ce qu'on veut pour un impayé ou un abus présumé, où la décision peut être revue.

Sprint 9 — Exploitation du cluster et statut public

Q9.1 — [PIÈGE] Pourquoi `/api/status` est-il public alors que `/api/admin/cluster/**` est protégé ?

Niveau : [PIÈGE] · Domaine : Sécurité

Ce que le jury vérifie : comprends-tu ce que tu exposes ?

Réponse [CODE]** : > **Ce sont deux besoins opposés. La page de statut public répond à la question « le service est-il disponible ? », que se pose un client** avant même de se connecter** — voire quand il n’y arrive pas. L’exiger authentifiée n’aurait aucun sens : c’est précisément quand la plateforme va mal qu’on la consulte.

Les endpoints `/api/admin/cluster/**` exposent au contraire des informations d’infrastructure — nœuds, namespaces, pods, stockage, événements — qui n’ont aucune raison d’être publiques.

Trois protections sur l’endpoint public : 1. Seul `GET` est autorisé (`.requestMatchers(HttpMethod.GET, "/api/status/**").permitAll()`) 2. Un rate limit par IP : `StatusRateLimitFilter`, **30 requêtes par fenêtre de 60 secondes** 3. L’information exposée est agrégée — un état de service et une liste d’incidents, pas la topologie du cluster

Le point qui montre que tu as réfléchi : `StatusService` partage le même `KubernetesClient` que la supervision ADMIN. La différence n’est pas dans l’accès aux données, mais dans **ce qui est renvoyé**. C’est un choix de conception à défendre : le filtrage est applicatif.

Limite honnête : le rate limit est **en mémoire par instance**. Avec plusieurs répliques du backend, chacune aurait son propre compteur. Nous n’avons qu’une réplique, donc c’est cohérent aujourd’hui, mais ça ne passerait pas à l’échelle sans un stockage partagé.

Sprint 10 — Durcissement et sécurité

Q10.1 — [EXPERT] Expliquez votre RBAC. Le principe du moindre privilège est-il respecté ?

Niveau : [EXPERT] [PIÈGE] · Domaine : Sécurité / Kubernetes

Question très probable, et ton `rbac.yaml` contient déjà la réponse honnête.

Réponse [CODE]** : ** > Partiellement, et c'est documenté dans le fichier lui-même. Deux `ClusterRole` sont liés au `ServiceAccount` du backend :

- `platform-api-cluster-reader` — lecture des `nodes` et `events`
- `platform-backend-role` — le rôle principal, couvrant `Pods`, `services`, `deployments`, `namespaces`, les `CRD Knative Serving` et `Eventing`, les `KafkaTopic Strimzi`, `Pods/log`, les `PVC`, les `ResourceQuota` et les `NetworkPolicy`

Les deux faiblesses, à assumer avant qu'on te les oppose :

1. Portée cluster trop large. Le rôle accorde `create/update/delete` sur `Pods`, `services`, `deployments` et `namespaces` à l'échelle du cluster entier. Concrètement, le `ServiceAccount` du backend peut supprimer des ressources dans n'importe quel namespace — `kube-system`, `monitoring`, `jenkins`, `kafka`. C'est plus large que ce dont le code a besoin.

Le commentaire du fichier explique pourquoi ça n'a pas été réduit : restreindre des permissions sur un cluster vivant risque de casser un chemin de code non détecté par une revue statique. Le ticket de réduction est identifié mais non traité.

2. ServiceAccount default. Aucun `ServiceAccount` dédié n'a été créé ; le backend utilise le `default` du namespace `platform`. Un `ServiceAccount` dédié serait la bonne pratique.

Pourquoi un ClusterRole et pas un Role ? Parce que le backend crée dynamiquement de nouveaux namespaces. Un `Role` est limité à un namespace existant : il ne pourrait pas agir dans un namespace qui n'existe pas encore.

Ce qui joue en ta faveur : trois manques ont été détectés et corrigés par un audit systématique — `persistentvolumeclaims`, `resourcequotas` et `networkpolicies` étaient absents, et les fonctionnalités correspondantes échouaient silencieusement. Le fait de l'avoir trouvé prouve une démarche, pas un hasard.

Q10.2 — Expliquez votre NetworkPolicy

Niveau : [AVANCÉ] · Domaine : Réseau / Sécurité

Réponse [CODE]** : ** Elle est créée automatiquement pour chaque namespace tenant par `KnativeService.ensureNetworkPolicyExists()`, appelée en cascade depuis le déploiement.

Deux lignes portent tout le modèle :

`podSelector: {}` — un sélecteur **vide** ne signifie pas « aucun pod » mais « **tous** les pods du namespace ». Contre-intuitif, et c'est ce qui rend la politique applicable aux applications déployées plus tard.

`policyTypes: [Ingress, Egress]` — déclarer un type bascule ce sens en **refus par défaut**. Il n'y a pas de règle « bloquer tout » à écrire : le blocage est la conséquence de la déclaration.

Flux ouverts (les exceptions) :

Sens	Autorisé	Raison
Entrée	Même namespace	Les apps d'un client se parlent
Entrée	<code>kourier-system</code>	Sinon aucune requête externe n'arrive
Entrée	<code>knative-serving</code>	Activator et sondes de santé
Entrée	<code>monitoring</code>	Prometheus doit scraper
Sortie	Même namespace	Symétrique
Sortie	<code>kube-system</code> ports 53	DNS — sans ça, plus aucune résolution
Sortie	<code>kafka</code>	Bus d'événements partagé
Sortie	<code>knative-serving</code> , <code>kourier-system</code>	Retour vers le plan de contrôle

Bloqué : le trafic direct entre deux tenants, et l'accès aux composants internes du namespace `platform` (PostgreSQL, Keycloak).

Le pattern « ensure » : la méthode est idempotente — si la politique existe, elle retourne immédiatement. Nécessaire, car `deploy()` est appelée à chaque déploiement, pas seulement au premier.

Limite documentée : introduite en cours de projet, elle ne couvre pas les namespaces créés avant, faute de reprise rétroactive. La correction serait un job au démarrage parcourant les namespaces `user-*` — le code existe déjà et il est idempotent, il ne manque que la boucle.

Q10.3 — Cilium est-il un mécanisme d'authentification ?

Niveau : [PIÈGE] · **Domaine** : Sécurité / Réseau

Réponse : > Non, et la distinction est essentielle. Cilium opère au **niveau réseau** : il décide si un paquet peut circuler entre deux pods. Il ne sait rien de l'identité de l'utilisateur, ni de son rôle, ni de ce qu'il demande.

L'authentification et l'autorisation sont **applicatives** : JWT validé par Spring Security, rôle extrait du jeton, permission vérifiée par `PermissionService`, propriété vérifiée par `requireOwned`.

Les deux sont complémentaires et non substituables : un attaquant qui contournerait la couche applicative resterait bloqué par la NetworkPolicy ; un attaquant réseau resterait bloqué par le contrôle applicatif. C'est de la défense en profondeur.

Sprint 11 — Pipeline CI/CD complet

Q11.1 — Un développeur pousse un commit. Que se passe-t-il ?

Niveau : [INTERMÉDIAIRE] · Domaine : DevOps

Réponse [CODE]** : Le pipeline `platform-backend` enchaîne quatre stages :

1. **Cleanup** — supprime les workspaces `@script` obsolètes et les répertoires JNA de `/tmp`, puis recrée les répertoires temporaires persistants.
2. **Checkout** — clone la branche `main`, encadré par `retry(3)`, avec le credential `github-credentials`.
3. **Build JAR** — `mvn clean package -DskipTests` dans `backend-api`, avec le JDK 21.
4. **Docker Build & Push + Deploy** — Kaniko construit l'image depuis `backend-kaniko.Dockerfile` et publie sous deux étiquettes (`v${BUILD_NUMBER}` et `latest`), puis `kubectrl set image` met à jour le déploiement `platform-api` dans le namespace `platform`, et `kubectrl rollout status` vérifie l'aboutissement.

Un bloc `post` gère succès, échec, et déclenche systématiquement un `safeRestart` de Jenkins.

Q11.2 — [AVANCÉ] Pourquoi Kaniko plutôt que Docker-in-Docker ?

Niveau : [AVANCÉ] · Domaine : DevOps / Sécurité

Réponse : > Parce que construire une image **depuis l'intérieur d'un cluster Kubernetes** pose un problème de sécurité. Docker-in-Docker exige un conteneur **privilegié** ou le montage du socket Docker de l'hôte — dans les deux cas, une évaison de conteneur donne un accès root au nœud. C'est un risque inacceptable sur un cluster qui héberge aussi les applications de clients.

Kaniko construit l'image **en espace utilisateur**, sans démon Docker ni privilège root. Chaque instruction du Dockerfile est exécutée et un instantané du système de fichiers est pris.

Le prix payé — et c'est ta difficulté technique la plus originale : Kaniko efface `/tmp` pendant la construction, ce qui détruit le processus auxiliaire de Jenkins (`fork()` échoue). Trois parades ont été nécessaires [CODE] : - déport des répertoires temporaires de la JVM hors de `/tmp` via `JAVA_TOOL_OPTIONS` et `TMPDIR` - options `--use-new-run` et `--snapshot-mode=redo` - `safeRestart` de Jenkins en fin de pipeline

Q11.3 — [PIÈGE] Pourquoi déployer avec le tag versionné et pas `latest` ?

Niveau : [PIÈGE] · Domaine : DevOps / Kubernetes

Réponse : > Parce qu'avec `latest`, **Kubernetes ne détecterait aucun changement.** `kubectl set image` compare la spécification : si le nom de l'image reste `platform-api:latest`, le champ est identique, aucun nouveau ReplicaSet n'est créé, **aucun rollout n'est déclenché.** Les pods continueraient de tourner avec l'ancienne image.

Le tag `v${BUILD_NUMBER}` change à chaque build, ce qui force un vrai rollout. Il apporte aussi la traçabilité — on sait exactement quelle version tourne — et rend le retour arrière possible.

Nous publions quand même `latest` en parallèle, mais ce n'est pas ce qui est déployé.

Q11.4 — [EXPERT] Où sont vos secrets, et comment sont-ils protégés ?

Niveau : [EXPERT] · **Domaine :** Sécurité

Réponse [CODE]**.* > À trois niveaux, aucun en clair dans le dépôt :

Jenkins — `github-credentials` et `dockerhub-credentials` sont dans le magasin de credentials, injectés par `withCredentials()`. Jenkins masque automatiquement leur valeur dans les logs.

Kubernetes — le déploiement backend référence des `secretKeyRef` pour le mot de passe PostgreSQL, les identifiants admin Keycloak et les clés Stripe.

Configuration — les valeurs sensibles sont des variables d'environnement avec valeur par défaut vide : `${APP_JWT_SECRET:}`, `${STRIPE_SECRET_KEY:sk_test_REPLACE_ME}`.

Limite à reconnaître : les Secrets Kubernetes ne sont encodés qu'en **base64**, pas chiffrés au repos par défaut. Quelqu'un ayant accès à etcd ou le droit de lire les secrets peut les récupérer. Les solutions existent — chiffrement au repos, Sealed Secrets, ou un gestionnaire externe type Vault — mais aucune n'est en place. **[NON IMPLÉMENTÉ]**

Partie III — Questions transversales

Q-T1 — [EXPERT] Pourquoi dites-vous que votre plateforme est Cloud Native ?

Niveau : [EXPERT] · Domaine : Architecture

Réponse : > Parce qu'elle réunit les caractéristiques qui définissent le terme, et pas seulement parce qu'elle tourne sur Kubernetes :

- **Conteneurisation** — tout est packagé en images
- **Orchestration déclarative** — on décrit l'état désiré, Kubernetes réconcilie
- **Élasticité** — mise à l'échelle automatique jusqu'à zéro via Knative
- **Découplage par événements** — Kafka et Knative Eventing
- **Extension par CRD et opérateurs** — Knative, Strimzi
- **Automatisation du déploiement** — pipelines CI/CD
- **Observabilité** — Prometheus, logs temps réel

La nuance qui te distingue : il ne suffit pas de faire tourner une application dans un conteneur sur Kubernetes pour être Cloud Native. Ce qui compte, c'est de concevoir **pour** cet environnement : accepter qu'un pod puisse disparaître, être sans état, déclarer ses ressources. Notre backend le fait — il est **STATELESS**, tout l'état est en base ou dans le cluster.

Où nous ne le sommes pas complètement : l'infrastructure elle-même n'est pas décrite en code (pas d'IaC), et le backend tourne en instance unique.

Q-T2 — [EXPERT] Où se situe la frontière de confiance de votre architecture ?

Niveau : [EXPERT] · Domaine : Sécurité

Réponse : Il y en a plusieurs, emboîtées :

1. **Externe → cluster** — franchie par MetalLB puis Kourier. Tout ce qui entre est non fiable.
2. **Non authentifié → authentifié** — franchie par la validation du JWT. Frontière la plus nette.
3. **Authentifié → autorisé** — permission puis propriété de la ressource.
4. **Namespace platform → namespace tenant** — la NetworkPolicy. Le code client ne doit jamais atteindre PostgreSQL ni Keycloak.
5. **Tenant → tenant** — la frontière la plus critique commercialement.

Le point le plus sensible : le backend est **l'unique point d'entrée vers le cluster**. Cela centralise les contrôles — un avantage — mais concentre aussi le risque : une compromission du backend donnerait accès à son ClusterRole, donc à un pouvoir étendu sur le cluster. C'est le prix de l'architecture, et il faut savoir le nommer.

Q-T3 — [EXPERT] Quel est le principal point de défaillance unique ?

Niveau : [EXPERT] · Domaine : Architecture

Réponse : Plusieurs, par ordre de gravité :

Composant	Impact si perte	État
PostgreSQL	Perte de données, backend inutilisable	Instance unique, pas de réplication
Backend	Plus aucune action possible sur la plateforme	replicas: 1 [CODE]
Control-plane	Plus de déploiement, plus de scale-to-zero	Un seul nœud
Keycloak	Plus de connexion possible ; jetons en cours valides	Instance unique
Broker Kafka	Messages non consommés perdus	Un seul broker

Le plus critique est PostgreSQL : c'est le seul composant dont la perte est irréversible. Le reste peut être redémarré.

Le détail expert : le backend ne peut pas être simplement répliqué. KnativeWatcher et les jobs @Scheduled s'exécuteraient en double — facturation comptée deux fois, alertes dupliquées. Il faudrait d'abord un verrou distribué, par exemple ShedLock. [NON IMPLÉMENTÉ] Ce point précis montre que tu as réfléchi au passage à l'échelle réel.

Q-T4 — Comment votre architecture évolue-t-elle de 10 à 1 000 clients ?

Niveau : [EXPERT] · Domaine : Architecture

Réponse structurée :

Ce qui tient sans changement de conception : le modèle multi-tenant lui-même. L'isolation par namespace, la résolution du contexte utilisateur et les quotas ne font aucune hypothèse sur le nombre de tenants. C'est l'infrastructure qu'il faudrait renforcer, pas l'architecture logicielle à repenser — c'est la distinction importante à faire valoir.

Ce qui casse en premier, dans l'ordre : 1. La capacité des 3 nœuds — plafond mécanique sur le nombre de pods 2. PostgreSQL — instance unique, pas de réplique de lecture 3. Le backend — instance unique, et non répliquable en l'état (voir Q-T3) 4. Kafka — un broker 5. Le nombre de namespaces — Kubernetes gère des milliers de namespaces, mais chaque NetworkPolicy ajoute des règles à évaluer

Ce qu'il faudrait : nœuds supplémentaires, PostgreSQL en réplication, backend répliqué avec verrou distribué, Kafka à 3+ brokers, et une réflexion sur le partitionnement en plusieurs clusters au-delà d'un certain seuil.

Ne surtout pas inventer de chiffre de capacité. Nous n'avons pas fait de test de charge.

Q-T5 — Comment prouveriez-vous que deux tenants sont réellement isolés ?

Niveau : [EXPERT] · Domaine : Sécurité

Réponse — un protocole de test en quatre volets :

Test réseau : depuis un pod du tenant A, tenter d'atteindre l'IP d'un pod du tenant B (`kubectl exec ... -- curl <ip-pod-B>`). Le résultat attendu est un timeout, pas un refus — la NetworkPolicy fait tomber les paquets silencieusement.

Test applicatif : avec le jeton du client A, appeler `GET /api/apps/{id-de-B}` . Résultat attendu : **403**, produit par `requireOwned` .

Test de quota : dépasser le nombre d'applications autorisé. Résultat attendu : **409** avec message explicite.

Test de contrôle : vérifier qu'un flux *autorisé* fonctionne (ex. résolution DNS depuis un pod tenant). Sans ce test, on ne distingue pas une isolation correcte d'un réseau cassé.

L'honnêteté qui compte : ces tests ont été menés ponctuellement, mais **il n'existe pas de suite automatisée** qui les rejoue. C'est exactement ce que je mettrais en place en priorité.

Q-T6 — Où est votre dette technique ?

Niveau : [EXPERT] · Domaine : Recul

Réponse — sois précis, c'est ce qui impressionne :

Dette	Nature	Gravité
Flow ROPC au lieu de PKCE	Sécurité	Élevée
RBAC à portée cluster non réduite	Sécurité	Élevée
Aucun test automatisé	Qualité	Élevée
Pas d'IaC pour l'infrastructure	Exploitabilité	Élevée
ServiceAccount <code>default</code> au lieu d'un dédié	Sécurité	Moyenne
NetworkPolicy absente sur les namespaces anciens	Sécurité	Moyenne
Webhook Stripe non idempotent	Fiabilité	Moyenne
<code>backend.Dockerfile</code> code mort	Hygiène	Faible
<code>keycloak.js</code> code mort	Hygiène	Faible
Dépendance Redis déclarée mais inutilisée	Hygiène	Faible

Le fait de pouvoir énumérer cette liste est en soi une réponse forte. Elle montre que tu as audité ton propre travail.

Q-T7 — Que se passe-t-il si... (scénarios de panne)

Niveau : [AVANCÉ] à [EXPERT]

Scénario	Comportement réel de notre projet
Un pod tombe	Knative/Kubernetes le recrée. KnativeWatcher met le statut à jour.
Un worker tombe	Les pods sont replanifiés sur l'autre worker, si la capacité suffit.
Le control-plane tombe	Les apps continuent ; plus aucun déploiement ni réveil possible.
PostgreSQL tombe	Le backend ne répond plus sur les endpoints métier. Les apps déployées continuent — elles ne dépendent pas de notre base.
Keycloak tombe	Impossible de se connecter. Les jetons déjà émis restent valides jusqu'à expiration : la validation est locale, par signature.
Kafka tombe	Création de topics et publication en échec. Les apps déjà déployées tournent, mais ne sont plus réveillées par événement.
Le broker Kafka est perdu	Messages non consommés perdus — un seul broker, réplication impossible.
JWT expiré	401, puis rafraîchissement automatique côté portail.
JWT falsifié	Rejeté à la validation de signature.
Image inexistante	Le pod passe en ImagePullBackOff , KnativeWatcher remonte FAILED , l'utilisateur voit l'échec dans ses journaux.
Rollout échoue	kubectl rollout status échoue → le build Jenkins échoue. Aucun rollback automatique — l'ancienne version reste en place.
Git indisponible	retry(3) réessaie, puis le pipeline échoue.
Docker Hub indisponible	Le push Kaniko échoue, aucun déploiement.

Q-T8 — Questions de debugging

Niveau : [AVANCÉ] · Domaine : Exploitation

Symptôme	Diagnostic	Cause probable
CrashLoopBackOff	<code>kubectl logs <pod> --previous</code>	L'app plante au démarrage. Notre <code>CrashLoopScheduler</code> alerte après 5 redémarrages.
ImagePullBackOff	<code>kubectl describe pod</code>	Image inexistante, tag erroné, ou registre inaccessible.
Pending	<code>kubectl describe pod</code> → section Events	Pas de nœud avec assez de ressources, ou ResourceQuota du tenant atteint .
OOMKilled	<code>kubectl describe pod</code> → Last State	Limite mémoire dépassée. Chez nous, 128Mi par défaut — insuffisant pour beaucoup d'apps Java.
Service inaccessible	<code>kubectl get ksvc</code> → colonne READY	Révision non prête, ou <code>RevisionMissing</code> .
DNS ne résout pas	<code>kubectl exec -- nslookup</code>	Suspect n°1 : la NetworkPolicy. Si la règle egress vers <code>kube-system:53</code> manque, plus rien ne résout.
Trafic bloqué	<code>cilium status</code> , tester depuis un pod	NetworkPolicy trop restrictive.
Kafka ne reçoit rien	<code>kubectl exec -- kafka-topics --list</code>	Mauvais bootstrap server, ou NetworkPolicy bloquant l'egress vers <code>kafka</code> .
Trigger ne déclenche pas	<code>kubectl get trigger</code> → READY	Broker et Trigger dans des namespaces différents — notre erreur historique.
JWT rejeté	Logs Spring Security en DEBUG	<code>issuer-uri</code> incorrect, jeton expiré, ou horloges désynchronisées.
403 inattendu	Vérifier le rôle et les permissions en base	Permission <code>MEMBER</code> absente, ou <code>requireOwned</code> sur une ressource d'un autre.
Kaniko échoue	Logs Jenkins du stage	Le problème <code>/tmp</code> documenté — vérifier <code>TMPDIR</code> et <code>JAVA_TOOL_OPTIONS</code> .

Partie IV — 30 questions pièges

Dans chacune, le jury affirme quelque chose de **partiellement faux**. La bonne réponse commence par nuancer.

1. « **Vous utilisez Kubernetes, donc votre application est hautement disponible ?** » Non. Kubernetes fournit les mécanismes, pas la configuration. Chez nous : `replicas: 1`, un control-plane, un broker Kafka, une instance PostgreSQL.
2. « **Un namespace garantit l'isolation ?** » Non. Il isole les noms, pas le réseau ni les ressources. Il faut NetworkPolicy et ResourceQuota par-dessus.
3. « **JWT signifie que votre système est sécurisé ?** » Non. Le JWT résout l'authentification. Il ne dit rien de l'autorisation, ni de la propriété d'une ressource. C'est justement l'objet de `requireOwned`.
4. « **Kafka garantit qu'aucun message n'est perdu ?** » Pas chez nous : un seul broker, donc pas de réplication.
5. « **Knative, donc vous êtes serverless ?** » Nos applications clientes le sont. Notre plateforme elle-même — backend, PostgreSQL, Keycloak — tourne en permanence.
6. « **latest garantit la dernière version ?** » Non, l'inverse. Déployer avec `latest` ne déclenche aucun rollout, car la spécification ne change pas.
7. « **Votre frontend cache un bouton, donc l'utilisateur n'y a pas accès ?** » Non. Le frontend n'est pas une frontière de sécurité. Tout est vérifié côté backend.
8. « **Docker Hub suffit à sécuriser votre supply chain ?** » Non. Nous ne signons pas les images et ne les scannons pas. C'est une limite assumée.
9. « **Vos images sont dans un registre public, c'est un risque ?** » Oui, il faut le reconnaître. Un registre privé serait préférable en production.
10. « **Vos secrets Kubernetes sont chiffrés ?** » Non, encodés en base64 seulement. Pas de chiffrement au repos configuré.
11. « **Le scale-to-zero, c'est gratuit ?** » Non. Le prix est le cold start. Et nous facturons un coefficient résiduel de 0,2.
12. « **Cilium sécurise vos utilisateurs ?** » Non. Cilium sécurise le **réseau**. Les utilisateurs sont sécurisés par Keycloak et Spring Security.
13. « **Vous avez des tests ?** » Non, aucun test automatisé. La validation a été manuelle, avec Postman et vérification directe sur le cluster.
14. « **Votre CI/CD est complète ?** » Non. Elle construit et déploie, mais sans tests, sans analyse statique, sans scan de vulnérabilités, sans rollback automatique.
15. « **Un ADMIN peut gérer les équipes des clients ?** » Non. `TeamController` est réservé à `CLIENT_ADMIN`. L'ADMIN n'y a pas accès.
16. « **Vos quotas sont appliqués par le backend ?** » Aux deux niveaux : applicatif (409 avant création) **et** Kubernetes (ResourceQuota). La seconde barrière tient même si le backend est contourné.

- 17. « Votre backend est un opérateur Kubernetes ? »** Non. Il est client de l'API. `KnativeWatcher` observe, il ne réconcilie pas.
- 18. « Le SSE, c'est comme le WebSocket ? »** Non. SSE est unidirectionnel, sur HTTP standard, avec reconnexion automatique native.
- 19. « Vous utilisez la CRD KafkaTopic de Strimzi ? »** Non, `AdminClient`. La classe `KafkaTopic` de notre code est une entité JPA, pas la CRD.
- 20. « Vos applications clientes consomment Kafka directement ? »** Non. Elles reçoivent des `CloudEvents` en HTTP. C'est la `KafkaSource` qui parle Kafka.
- 21. « Le rate limit protège votre page de statut ? »** Partiellement. Il est en mémoire, par instance. Avec plusieurs répliques, chacune aurait son compteur.
- 22. « Vous respectez le moindre privilège ? »** Partiellement. Le RBAC est à portée cluster, plus large que nécessaire, et documenté comme tel.
- 23. « Un MEMBER a son propre espace ? »** Non. Il travaille dans le namespace de son `CLIENT_ADMIN`, via `UserContextService.resolve()`.
- 24. « Supprimer une application efface tout ? »** Non. Le service Knative est détruit, mais l'entrée reste en base avec le statut `DELETED`, pour préserver l'historique de facturation.
- 25. « Vous facturez la consommation réelle ? »** Non, les ressources **demandées**, pondérées par un facteur de disponibilité. C'est le modèle des fournisseurs cloud.
- 26. « Le webhook Stripe est public, donc vulnérable ? »** Public oui, non authentifié non : la signature cryptographique remplace l'authentification.
- 27. « Vos webhooks sont idempotents ? »** Non. C'est une limite réelle : un événement rejoué serait traité deux fois.
- 28. « Un client peut voir les logs d'un autre ? »** Non. Le filtrage se fait par identité effective résolue côté serveur, jamais par l'identifiant de l'URL.
- 29. « Votre plateforme est prête pour la production ? »** Elle est fonctionnellement complète et opérationnelle. Elle n'est pas prête pour un usage commercial : haute disponibilité, tests, IaC et durcissement du RBAC manquent.
- 30. « Vous avez tout fait vous-même ? »** Le code applicatif, l'architecture et l'intégration, oui. Kubernetes, Knative, Strimzi et Keycloak sont des briques existantes — et c'était le bon choix : les réimplémenter aurait été absurde.
-

Partie V — Réponses orales à mémoriser

Ces réponses doivent sortir naturellement. Répète-les à voix haute.

« Présentez votre projet en une minute »

PlatformServerless est une plateforme PaaS multi-tenant qui permet à un client de déployer une application conteneurisée à partir d'une simple image Docker, sans connaître Kubernetes. L'exécution est serverless : l'application descend à zéro instance quand elle n'est pas sollicitée, remonte automatiquement à la première requête, et n'est facturée que sur sa consommation réelle. Elle s'intègre nativement à un bus d'événements Kafka, ce qui permet de réveiller une application sur un événement métier et pas seulement sur une requête HTTP. Techniquement, c'est un backend Spring Boot qui pilote un cluster Kubernetes via Fabric8, avec Knative pour le serverless, Strimzi pour Kafka, Keycloak pour l'identité, et deux interfaces React.

« Présentez votre architecture »

Quatre couches. Une couche présentation avec deux applications React — le portail client et la console d'administration. Une couche service, le backend Spring Boot, qui est le **point d'entrée unique** vers le cluster : aucun frontend ne parle directement à Kubernetes. Une couche infrastructure, le cluster à trois nœuds avec Knative pour l'exécution serverless et Kafka opéré par Strimzi. Et une couche persistance, PostgreSQL pour l'état applicatif. L'isolation multi-tenant traverse toutes ces couches : un namespace par client, une NetworkPolicy default-deny, et une vérification de propriété sur chaque accès à une ressource.

« Quel est votre apport personnel ? »

L'architecture d'ensemble et l'intégration. Les briques existent — Kubernetes, Knative, Strimzi, Keycloak — mais les faire fonctionner ensemble en un produit cohérent, avec un modèle multi-tenant sûr, une facturation reflétant l'usage réel et une chaîne de déploiement automatisée, c'est le travail que j'ai mené. J'ai aussi audité mon propre projet, ce qui m'a permis de trouver des écarts réels — des droits RBAC manquants qui faisaient échouer silencieusement des fonctionnalités, une fuite de jeton dans les paramètres d'URL — et de les corriger.

« Votre choix technique le plus important ? »

Knative. Il conditionne tout le reste : le scale-to-zero, le modèle de facturation à l'usage, le rollback par révisions, et la capacité à réveiller une application sur événement. Sans lui, il aurait fallu réimplémenter tout ça, ce qui aurait été un projet à part entière.

« Votre plus grande difficulté technique ? »

L'intégration de Knative Eventing. Trois raisons. D'abord il n'existe **aucun client Java officiel** pour Knative : il a fallu construire les manifestes comme des structures non typées via l'API générique de Fabric8. Ensuite une contrainte non documentée clairement : un Trigger et son Broker doivent être dans le même namespace, sinon le Trigger reste silencieusement non fonctionnel — pas d'erreur, l'événement disparaît simplement. Enfin la chaîne KafkaSource → Broker → Trigger → Service comporte plusieurs points de rupture qui ne produisent aucune erreur franche. Le diagnostic s'est fait maillon par maillon avec `kubectl get`.

« Si vous refaisiez le projet, que changeriez-vous ? »

Trois choses, par ordre d'importance. **Le flow d'authentification** : Authorization Code + PKCE dès le départ au lieu de ROPC — c'est le seul choix qui touche à la sécurité du produit lui-même et qui conditionne le SSO et le MFA. **L'Infrastructure as Code** : décrire le cluster dans le dépôt dès le Sprint 0 ; sans ça la plateforme n'est pas reproductible. **Les tests automatisés**, au minimum sur la logique de facturation, qui est du calcul pur et dont une erreur aurait une conséquence directe pour le client.

« Expliquez la sécurité de votre plateforme »

Elle est en couches. Le réseau d'abord : Cilium applique une NetworkPolicy default-deny qui empêche deux clients de communiquer. L'authentification ensuite : Keycloak émet un JWT, que le backend valide par signature sans jamais rappeler Keycloak. L'autorisation à deux niveaux : le rôle donne accès à un type d'action, la permission granulaire filtre les membres d'équipe, et une vérification de propriété s'assure que la ressource appartient bien à l'appelant. Les secrets ne sont jamais en clair. Et je connais les limites : le flow ROPC, un RBAC plus large que nécessaire, et pas de scan de vulnérabilités.

Partie VI — Tableaux de synthèse

| Toutes les technologies

Technologie	Rôle réel chez nous	Pourquoi	Alternative	Question difficile
Kubernetes	Orchestration, isolation par namespace	Seul à offrir namespaces + CRD + ordonnanceur	Docker Swarm, Nomad	« Donc vous êtes HA ? »
Cilium 1.19.1	CNI + NetworkPolicy	eBPF, évolutivité	Calico, Flannel	« Cilium authentifie ? »
MetalLB	IP externes on-premise	Absent en bare-metal	NodePort seul	« Et en cloud ? »
Kourier	Passerelle Knative	Ingress par défaut de Knative	Istio, Contour	« Pourquoi pas Istio ? »
Knative Serving	Scale-to-zero, révisions	Cœur du modèle serverless	OpenFaaS, KEDA	« Et le cold start ? »
Knative Eventing	Broker, Trigger, CloudEvents	Réveil sur événement	Consommateur Kafka classique	« Pourquoi un Broker ? »
Kafka	Bus d'événements	Standard, intégré à Knative	RabbitMQ, NATS	« Message perdu ? »
Strimzi 0.45.0	Opérateur Kafka	Kafka natif sur K8s	Kafka manuel	« ZooKeeper ou KRaft ? »
Keycloak	Fournisseur d'identité	OIDC standard, évite l'auth maison	Auth0, auth maison	« Quel flow OAuth2 ? »
PostgreSQL	État applicatif	Relationnel, transactions	MySQL, MongoDB	« Et si elle tombe ? »
Fabric8 6.10.0	Client Java K8s	Typé, watch, erreurs	kubectl en subprocess	« Pourquoi pas kubectl ? »
Spring Boot 3.2.3	Backend	Écosystème sécurité mature	Quarkus, Micronaut	« Pourquoi pas natif ? »
React + Vite	Portail et console	SPA, écosystème	Angular, Vue	« Duplication de code ? »
Jenkins	CI/CD	Auto-hébergé, maîtrisé	GitHub Actions, GitLab CI	« Pourquoi pas Actions ? »
Kaniko	Build sans démon Docker	Pas de conteneur privilégié	Docker-in-Docker, Buildah	« Le problème / tmp ? »
Docker Hub	Registre	Simple, gratuit	Registre privé, Harbor	« Supply chain ? »
Prometheus	Métriques	Standard K8s	Datadog	« Rétention ? »
Stripe	Paieement	Standard, PCI délégué	PayPal	« Signature webhook ? »

Checklist « je sais expliquer »

Coche uniquement quand tu peux répondre **sans notes**, à voix haute.

Élément	Son rôle	Notre implémentation	Pourquoi ce choix
Kubernetes	[]	[]	[]
Cilium / NetworkPolicy	[]	[]	[]
MetallB	[]	[]	[]
Kourier	[]	[]	[]
Knative Serving	[]	[]	[]
Knative Eventing	[]	[]	[]
Kafka	[]	[]	[]
Strimzi	[]	[]	[]
Keycloak / JWT	[]	[]	[]
PostgreSQL	[]	[]	[]
Fabric8	[]	[]	[]
Jenkins / Kaniko	[]	[]	[]
Multi-tenance	[]	[]	[]
Rôles et permissions	[]	[]	[]
Facturation	[]	[]	[]
Observabilité (SSE)	[]	[]	[]
Administration et audit	[]	[]	[]
RBAC	[]	[]	[]

Les 12 questions à préparer en priorité

Si le temps manque, concentre-toi sur celles-ci. Les quatre premières tombent presque à coup sûr, dans les premières minutes.

#	Question	Pourquoi elle compte
1	Q-A1 — Ta problématique	Première question de toute soutenance. Décide de la première impression.
2	Q-A2 — Étude de l'existant	« Pourquoi ne pas avoir pris Heroku ou Cloud Run ? » — inévitable.
3	Q-A4 — Valeur ajoutée	Teste si tu penses produit ou seulement technique.
4	Q-A5 — Architecture événementielle	Concept central du projet, souvent mal maîtrisé.
5	Q3.1 — Flow OAuth2 (ROPC)	La plus dangereuse techniquement.
6	Q4.1 — Tracer un déploiement de bout en bout	Le cas d'utilisation central du projet.
7	Q3.4 — IDOR et <code>requireOwned</code>	Le cœur de l'isolation multi-tenant.
8	Q5.1 — Chemin d'un événement Kafka	La question technique la plus riche.
9	Q4.2 — Le piège du proxy Spring	Ta meilleure preuve de maîtrise.
10	Q10.1 — RBAC et moindre privilège	Ton <code>rbac.yaml</code> contient déjà la réponse honnête.
11	Q-T3 — Point de défaillance unique	Teste ton recul architectural.
12	Q-T6 — Ta dette technique	Presque toujours posée, sous une forme ou une autre.

Trois principes pour le jour J

Assume tes choix, ne les invente pas. Un « je n'ai pas implémenté ça, voici pourquoi et ce que je ferais » vaut toujours mieux qu'une réponse floue qui s'effondre à la relance.

Devance les faiblesses que tu connais. Ce guide en liste une dizaine. Les mentionner toi-même te fait passer d'un candidat pris en défaut à un ingénieur qui a audité son travail.

Ramène toujours à ton code. À chaque question théorique, enchaîne sur « dans notre projet, concrètement... ». C'est ce qui distingue quelqu'un qui connaît son mémoire de quelqu'un qui connaît son projet.

Toutes les affirmations marquées **[CODE]** ont été vérifiées dans le code source, les fichiers de configuration ou les manifestes Kubernetes du projet. Aucune donnée, mesure ou fonctionnalité n'a été inventée.